DiffuChannel benchmark: diffusion vs DeepGANnel across all phenotypes

Barrett-Jolley lab

07 August 2026

Contents

Purpose

One pass over every phenotype folder. For each one it reads the real seed record, the DeepGANnel record and the diffusion (DDPM) record, computes the same metric set on the two synthetic arms, and then pools the five paired comparisons.

This replaces `DiffuChannel_analysis.Rmd` (one seed at a time, file dialogs) and `DiffuChannel_meta.Rmd` (pooling) with a single non-interactive knit. The metric definitions are carried over unchanged so numbers stay comparable with anything already computed.

Every arm is scored identically. Whether that flatters the diffusion model is not this notebook's business.

```
required <- c("dplyr", "tidyr", "ggplot2", "readr", "purrr", "arrow",
              "scales", "patchwork", "tibble")
missing  <- required[!vapply(required, requireNamespace, logical(1), quietly = TRUE)]
if (length(missing)) {
  stop("Install first: install.packages(c(\"",
       paste(missing, collapse = "\", \""), "\"))")
}

library(dplyr)
library(tidyr)
library(ggplot2)
library(readr)
library(purrr)
library(arrow)
library(patchwork)

set.seed(42)
```

Configuration

```
CFG <- list(
  # ---- where things live ---------------------------------------------------
  pheno_root = "./Phenotypes",    # holds Phenotype_A, Phenotype_B, ...
  batch_dir  = "./batch_out",     # holds Phenotype_A_DDPM.parquet, ...
  out_dir    = "./benchmark_out",

  raw_pattern = "raw",            # substring marking the seed csv (case-insensitive)
  gan_pattern = "gan",            # substring marking the DeepGANnel csv
  only        = NULL,             # e.g. c("Phenotype_A") to run a subset

  # ---- recording parameters ------------------------------------------------
  sample_hz  = 1e4,               # fallback only; a Time column always wins
  filter_hz  = 1e3,
```

```
  # ---- how to treat a generated file's own time column --------------------
  #   "seed"  every arm inherits the raw record's rate (treat as equivalent:
  #           the generated file's time column is cosmetic and wrong)
  #   "own"   an arm with a time column is taken at its word (treat as honest:
  #           it really was produced at that rate)
  rate_policy  = "seed",
  rate_compare = TRUE,           # also run the other policy where they differ

  # ---- must match training ------------------------------------------------
  seq_len     = 128,
  slow_factor = 64,

  # ---- analysis knobs -----------------------------------------------------
  n_windows_mmd = 400,           # O(n^2) in the kernel: keep modest
  n_perm        = 200,
  psd_seg       = "auto",        # "auto" = largest power of 2 every record supports
  psd_seg_min   = 256,           # below this, stop rather than report a coarse PSD
  acf_lag_max   = 200,
  # Which conductance level supplies the noise segments. "auto" picks, per
  # phenotype, whichever of closed or open every arm can support; a record that
  # barely closes has no baseline but plenty of open-level material.
  noise_level   = "auto",        # "auto", "closed" or "open"
  ks_subsample  = 2e5,
  trace_secs    = 0.3,

  # ---- optional extras ----------------------------------------------------
  do_manifold    = FALSE,        # UMAP / t-SNE, needs uwot + Rtsne, slow
  n_windows_proj = 400,
  proj_win       = 512,

  cache = TRUE                   # reuse per-phenotype results if present
)

dir.create(CFG$out_dir, showWarnings = FALSE, recursive = TRUE)
dir.create(file.path(CFG$out_dir, "cache"), showWarnings = FALSE, recursive = TRUE)

pal <- c("Real" = "#1b1b1b", "Diffusion" = "#c0392b", "GAN" = "#8e44ad",
         "Real floor" = "#7f8c8d", "White noise" = "#bdc3c7")

theme_channel <- function(base_size = 10) {
  theme_classic(base_size = base_size) +
    theme(strip.background = element_blank(),
          strip.text = element_text(face = "bold", hjust = 0),
          legend.position = "top",
          legend.title = element_blank(),
          legend.key.size = unit(0.8, "lines"),
          axis.text = element_text(colour = "black"))
}

save_fig <- function(plot, name, width = 8, height = 5) {
  ggsave(file.path(CFG$out_dir, paste0(name, ".pdf")), plot,
         width = width, height = height)
  ggsave(file.path(CFG$out_dir, paste0(name, ".png")), plot,
```

```
         width = width, height = height, dpi = 300)
  invisible(plot)
}
```

Reading records

The seed and GAN csv files were not written by this pipeline, so their headers cannot be assumed. Named columns are used when they are recognisable; otherwise the columns are inferred the same way `batch_run.py` infers them, which is the only way the R and Python sides can agree on which column was which. The resolved mapping is printed in the manifest below and is worth reading once.

```
normalise_names <- function(nms) gsub("[_-]", " ", trimws(tolower(nms)))

# 10%-of-range quantisation, as in batch_run.py stage_seed()
quantise <- function(x) {
  span <- diff(range(x, na.rm = TRUE))
  if (span > 0) round(x / (0.1 * span)) else rep(0, length(x))
}

# A channels column may hold no more than this many discrete levels (MAX_LEVELS
# in batch_run.py). Used both to infer the column and to check a named one.
MAX_LEVELS <- 10

is_time_col <- function(x) {
  d <- diff(x)
  length(d) > 1 && all(d > 0) &&
    isTRUE(all.equal(d, rep(d[1], length(d)), tolerance = 1e-3))
}

infer_cols <- function(df) {
  # The column count decides whether a time column exists at all: three columns
  # means one of them is time, two means there is none. Which one it is still
  # has to be found, because the order varies.
  time_col <- NA_integer_
  if (ncol(df) >= 3) {
    hit <- which(vapply(df, is_time_col, logical(1)))
    if (!length(hit)) {
      stop(ncol(df), " columns but none of them increases in constant steps, ",
           "so the time column cannot be identified")
    }
    time_col <- hit[1]
  }
  rest <- setdiff(seq_along(df), if (is.na(time_col)) integer(0) else time_col)
  lev  <- lapply(rest, function(j) sort(unique(quantise(df[[j]]))))
  n    <- vapply(lev, length, integer(1))
  k    <- which.min(n)
  if (n[k] > MAX_LEVELS) {
    stop("no column with <= ", MAX_LEVELS, " discrete levels; level counts ",
         paste(n, collapse = ", "))
  }
  others <- setdiff(seq_along(rest), k)
  list(time   = time_col,
```

```
       chan   = rest[k],
       cur    = rest[others[which.max(n[others])]],
       levels = lev[[k]])
}

read_record <- function(path) {
  df <- if (grepl("\\.parquet$", path, ignore.case = TRUE)) {
    as.data.frame(arrow::read_parquet(path))
  } else {
    d <- suppressWarnings(as.data.frame(
      readr::read_csv(path, show_col_types = FALSE, name_repair = "minimal")))
    # A file with no header gets its first data row promoted to names; spot that
    # by the names parsing as numbers, and re-read.
    if (all(!is.na(suppressWarnings(as.numeric(names(d)))))) {
      d <- as.data.frame(readr::read_csv(path, col_names = FALSE,
                                         show_col_types = FALSE))
    }
    d
  }

  df <- df[, vapply(df, function(x) is.numeric(x) || is.logical(x) ||
                      !all(is.na(suppressWarnings(as.numeric(as.character(x))))),
                    logical(1)), drop = FALSE]
  df[] <- lapply(df, function(x) suppressWarnings(as.numeric(as.character(x))))
  df <- df[stats::complete.cases(df), , drop = FALSE]
  if (ncol(df) < 2 || nrow(df) < 100) stop("too few usable rows/columns")

  nn  <- normalise_names(names(df))
  idx <- function(target) { h <- which(nn %in% target); if (length(h)) h[1] else NA_integer_ }
  i_chan <- idx(c("channels", "channel", "ideal", "idealisation", "idealization"))
  i_cur  <- idx(c("noisy current", "current", "raw", "raw current", "i"))
  i_time <- idx(c("time", "t"))

  # Headers are either absent or correct, so a recognised pair is taken at face
  # value. Anything else is inferred. Column order is never assumed either way.
  if (!is.na(i_chan) && !is.na(i_cur) && i_chan != i_cur) {
    how  <- "named columns"
    chan <- as.integer(round(df[[i_chan]]))
  } else {
    inf <- infer_cols(df)
    i_chan <- inf$chan; i_cur <- inf$cur; i_time <- inf$time
    how  <- "inferred"
    chan <- match(quantise(df[[i_chan]]), inf$levels) - 1L
  }

  out <- tibble::tibble(channels = as.numeric(chan),
                        current  = as.numeric(df[[i_cur]]))
  out$time <- if (!is.na(i_time)) as.numeric(df[[i_time]]) else
    (seq_len(nrow(out)) - 1) / CFG$sample_hz
  out <- dplyr::filter(out, is.finite(channels), is.finite(current))

  attr(out, "map") <- sprintf(
    "%s: chan=col%d cur=col%d time=%s", how, i_chan, i_cur,
```

```
    if (is.na(i_time)) sprintf("none, assuming %g Hz", CFG$sample_hz)
    else paste0("col", i_time))
  dplyr::select(out, time, channels, current)
}

fs_of <- function(d, fallback = CFG$sample_hz) {
  if (nrow(d) < 3) return(fallback)
  dt <- stats::median(diff(d$time), na.rm = TRUE)
  if (!is.finite(dt) || dt <= 0) fallback else 1 / dt
}
```

Metric functions

Carried over from `DiffuChannel_analysis.Rmd` without change.

```
make_windows <- function(x, win, n_max = Inf, stride = NULL) {
  n <- length(x)
  if (n < win) return(matrix(numeric(0), nrow = 0, ncol = win))
  if (is.null(stride)) stride <- max(1L, floor((n - win) / max(1L, min(n_max, n))))
  starts <- seq(1L, n - win + 1L, by = stride)
  if (length(starts) > n_max) starts <- sample(starts, n_max)
  t(vapply(starts, function(s) x[s:(s + win - 1L)], numeric(win)))
}

sq_dists <- function(A, B) {
  pmax(outer(rowSums(A^2), rowSums(B^2), "+") - 2 * tcrossprod(A, B), 0)
}

# Median heuristic: gamma = 1 / median pairwise squared distance. The fixed 1/L
# used during training underflows to zero on current in pA and pins the
# statistic to its finite-sample floor.
median_gamma <- function(X, Y, n_sub = 200) {
  A <- X[sample(nrow(X), min(n_sub, nrow(X))), , drop = FALSE]
  B <- Y[sample(nrow(Y), min(n_sub, nrow(Y))), , drop = FALSE]
  m <- stats::median(sq_dists(A, B))
  if (!is.finite(m) || m <= 0) 1 / ncol(X) else 1 / m
}

mmd_rbf <- function(X, Y, gamma = NULL, unbiased = TRUE) {
  m <- nrow(X); n <- nrow(Y)
  if (m < 2 || n < 2) return(NA_real_)
  if (is.null(gamma)) gamma <- median_gamma(X, Y)
  Kxx <- exp(-gamma * sq_dists(X, X))
  Kyy <- exp(-gamma * sq_dists(Y, Y))
  Kxy <- exp(-gamma * sq_dists(X, Y))
  if (unbiased) {
    (sum(Kxx) - sum(diag(Kxx))) / (m * (m - 1)) +
      (sum(Kyy) - sum(diag(Kyy))) / (n * (n - 1)) - 2 * mean(Kxy)
  } else {
    mean(Kxx) + mean(Kyy) - 2 * mean(Kxy)
  }
}
```

```
mmd_permutation_p <- function(X, Y, n_perm = CFG$n_perm, gamma = NULL) {
  if (is.null(gamma)) gamma <- median_gamma(X, Y)
  obs <- mmd_rbf(X, Y, gamma)
  Z <- rbind(X, Y); m <- nrow(X)
  null <- replicate(n_perm, {
    i <- sample(nrow(Z))
    mmd_rbf(Z[i[seq_len(m)], , drop = FALSE], Z[i[-seq_len(m)], , drop = FALSE], gamma)
  })
  list(mmd = obs, p = (1 + sum(null >= obs)) / (n_perm + 1))
}

# Noise segments are taken at a single conductance level, so that no gating step
# falls inside a segment. Runs are returned separately and never concatenated:
# gluing them together puts a step discontinuity at every join, and a step is
# broadband, lifting the high-frequency end of the spectrum and dragging the ACF
# towards zero in proportion to how often the record gates. That bias falls
# hardest on exactly the busy records the comparison is meant to include.
at_level <- function(d, level) {
  if (identical(level, "closed")) d$channels == 0 else d$channels >= max(d$channels)
}

level_runs <- function(d, level, min_len = 1L) {
  r <- rle(at_level(d, level))
  ends <- cumsum(r$lengths); starts <- ends - r$lengths + 1
  keep <- which(r$values & r$lengths >= min_len)
  lapply(keep, function(i) d$current[starts[i]:ends[i]])
}

longest_at <- function(d, level) {
  r <- rle(at_level(d, level))
  if (!any(r$values)) 0L else max(r$lengths[r$values])
}

# Welch across a list of runs: every segment from every run long enough to hold
# one contributes to a single average periodogram.
welch_psd <- function(runs, fs, nperseg) {
  if (is.numeric(runs)) runs <- list(runs)
  runs <- runs[vapply(runs, length, integer(1)) >= nperseg]
  if (!length(runs)) stop("no closed run holds ", nperseg, " samples")

  w <- 0.5 - 0.5 * cos(2 * pi * (seq_len(nperseg) - 1) / (nperseg - 1))  # Hann
  U <- sum(w^2)
  acc <- numeric(nperseg %/% 2 + 1)
  nseg <- 0L
  for (x in runs) {
    x <- as.numeric(x[is.finite(x)])
    for (s in seq(1L, length(x) - nperseg + 1L, by = nperseg %/% 2)) {
      seg <- x[s:(s + nperseg - 1L)]
      seg <- seg - mean(seg)
      X <- stats::fft(seg * w)[seq_len(nperseg %/% 2 + 1)]
      acc <- acc + (Mod(X)^2) / (fs * U)
      nseg <- nseg + 1L
    }
```

```
  }
  psd <- acc / nseg
  psd[-c(1, length(psd))] <- 2 * psd[-c(1, length(psd))]
  tibble::tibble(freq = seq(0, fs / 2, length.out = length(psd)), psd = psd,
                 n_seg = nseg) |>
    dplyr::filter(freq > 0)
}

# Length-weighted mean of the per-run ACFs, for the same reason.
acf_of <- function(runs, lag_max, fs) {
  if (is.numeric(runs)) runs <- list(runs)
  runs <- runs[vapply(runs, length, integer(1)) > lag_max]
  if (!length(runs)) stop("no closed run longer than ", lag_max, " samples")
  w <- vapply(runs, length, integer(1))
  A <- vapply(runs, function(x)
    stats::acf(x - mean(x), lag.max = lag_max, plot = FALSE,
               demean = TRUE)$acf[, 1, 1], numeric(lag_max + 1))
  tibble::tibble(lag_n  = seq_len(lag_max + 1),
                 lag_ms = 1000 * (seq_len(lag_max + 1) - 1) / fs,
                 acf    = as.numeric(A %*% w / sum(w)))
}

dwell_times <- function(ideal, fs) {
  r <- rle(as.integer(round(ideal)))
  tibble::tibble(level = r$values, n_sample = r$lengths,
                 dwell_ms = 1000 * r$lengths / fs) |>
    dplyr::mutate(state = ifelse(level == 0, "Closed", "Open"))
}

block_mean <- function(x, factor) {
  factor <- max(1L, as.integer(factor))
  nb <- length(x) %/% factor
  if (nb < 1) return(mean(x))
  colMeans(matrix(x[seq_len(nb * factor)], nrow = factor))
}

ks_safe <- function(a, b, n_max = CFG$ks_subsample) {
  a <- a[is.finite(a)]; b <- b[is.finite(b)]
  if (length(a) > n_max) a <- sample(a, n_max)
  if (length(b) > n_max) b <- sample(b, n_max)
  suppressWarnings(stats::ks.test(a, b))
}

# Window choice for the example traces, ported from _best_window in
# batch_run.py so the R figures and the Python previews pick the same way:
# prefer a stretch whose Popen sits in [lo, hi], else the one nearest the middle
# of that band. Taking the first n samples instead lands on whatever the record
# happens to open with, which for a sparse phenotype is often no events at all.
best_window <- function(chan, n, lo = 0.4, hi = 0.8) {
  if (length(chan) <= n) return(1L)
  cs <- c(0, cumsum(as.numeric(chan)))
  k  <- (cs[(n + 1):length(cs)] - cs[1:(length(cs) - n)]) / n
  inside <- which(k >= lo & k <= hi)
```

```r
  if (length(inside)) return(inside[ceiling(length(inside) / 2)])
  which.min(abs(k - 0.5 * (lo + hi)))
}

rms_diff <- function(a, b) sqrt(mean((a - b)^2))
```

1. Find the files

```r
find_one <- function(folder, pattern) {
  f <- list.files(folder, pattern = "\\.csv$", full.names = TRUE, ignore.case = TRUE)
  hit <- f[grepl(pattern, basename(f), ignore.case = TRUE)]
  if (!length(hit)) return(NA_character_)
  if (length(hit) > 1) {
    warning(folder, ": ", length(hit), " files match '", pattern,
            "', using ", basename(hit[1]))
  }
  hit[1]
}

folders <- list.dirs(CFG$pheno_root, recursive = FALSE)
if (!is.null(CFG$only)) folders <- folders[basename(folders) %in% CFG$only]
if (!length(folders)) stop("No phenotype folders in ", CFG$pheno_root)

parquets <- list.files(CFG$batch_dir, pattern = "\\.parquet$", full.names = TRUE)

manifest <- purrr::map_dfr(folders, function(fd) {
  tag <- gsub(" ", "_", basename(fd))
  ddpm <- parquets[grepl(paste0("^", tag), basename(parquets))]
  if (!length(ddpm)) ddpm <- parquets[grepl(tag, basename(parquets), fixed = TRUE)]
  tibble::tibble(
    phenotype = tag,
    raw  = find_one(fd, CFG$raw_pattern),
    gan  = find_one(fd, CFG$gan_pattern),
    ddpm = if (length(ddpm)) ddpm[1] else NA_character_
  )
})

knitr::kable(
  manifest |> dplyr::mutate(dplyr::across(c(raw, gan, ddpm), basename)),
  caption = "Manifest. Check that each row's three files belong to the same phenotype."
)
```

Table 1: Manifest. Check that each row's three files belong to the same phenotype.

phenotype	raw	gan	ddpm
Phenotype_A	4096PYRraw.csv	4096PYRGAN.csv	Phenotype_A_DDPM.parquet
Phenotype_B	4096edr01raw.csv	4096edr01GAN.csv	Phenotype_B_DDPM.parquet
Phenotype_C	4096lina11raw.csv	4096Lina2GAN_data.csv	Phenotype_C_DDPM.parquet

phenotype	raw	gan	ddpm
Phenotype_D	pvnraw.csv	pvnGAN.csv	Phenotype_D_DDPM.parquet
Phenotype_E	c035[NF=50Hz]0-32_4kraw.csv	c035[NF=50Hz]0-32_4kGAN.csv	Phenotype_E_DDPM.parquet

```
bad <- manifest |> dplyr::filter(is.na(raw) | is.na(ddpm))
if (nrow(bad)) {
  warning("Dropping phenotypes with no seed or no DDPM file: ",
          paste(bad$phenotype, collapse = ", "))
  manifest <- dplyr::filter(manifest, !is.na(raw), !is.na(ddpm))
}
if (!nrow(manifest)) stop("Nothing left to analyse.")
```

2. Pre-flight

Every record is read and checked before a single metric is computed, so that a record which cannot support the analysis stops the run here rather than becoming an `NA` that propagates quietly into the pooled tables.

All records are held in memory for the rest of the notebook. That is the price of reading each file exactly once; if it becomes a problem, run fewer phenotypes at a time with `CFG$only`.

```
recs <- purrr::map(seq_len(nrow(manifest)), function(i) {
  row <- manifest[i, ]
  out <- list(Real = read_record(row$raw), Diffusion = read_record(row$ddpm))
  if (!is.na(row$gan)) out$GAN <- read_record(row$gan)
  out
})
names(recs) <- manifest$phenotype

inventory <- purrr::imap_dfr(recs, function(arms, ph) {
  purrr::imap_dfr(arms, function(d, a) {
    closed <- d$current[d$channels == 0]
    open   <- d$current[d$channels > 0]
    tibble::tibble(
      phenotype = ph, arm = a, n = nrow(d),
      has_time = !grepl("time=none", attr(d, "map")),
      fs = fs_of(d),
      longest_closed = longest_at(d, "closed"),
      longest_open   = longest_at(d, "open"),
      popen = mean(d$channels > 0), max_level = max(d$channels),
      baseline = median(closed),
      unit_amp = median(open) - median(closed),
      cols = attr(d, "map"))
  })
})

# A record that has been min-max scaled, or left in the wrong units, has a
# unitary amplitude unlike its seed's. Every amplitude metric then compares
# units rather than distributions, and the arm loses on bookkeeping.
scale_check <- inventory |>
  dplyr::group_by(phenotype) |>
```

```
  dplyr::mutate(rel = unit_amp / unit_amp[arm == "Real"]) |>
  dplyr::ungroup() |>
  dplyr::filter(arm != "Real", rel < 1 / 3 | rel > 3)
if (nrow(scale_check)) {
  stop("Unitary amplitude differs from the seed by more than 3x in: ",
       paste(sprintf("%s/%s (%.3g vs %.3g)", scale_check$phenotype,
                     scale_check$arm, scale_check$unit_amp,
                     scale_check$unit_amp / scale_check$rel), collapse = "; "),
       ". That is a scaling or units problem, not a modelling result: the ",
       "amplitude metrics would compare units rather than distributions. ",
       "Check whether a min-max scaled copy has been used.")
}

knitr::kable(inventory, digits = 4,
             caption = "Pre-flight inventory. Every record that will be analysed.")
```

Table 2: Pre-flight inventory. Every record that will be analysed.

phenotype	arm	n	has_time	fs	longest_clone	longest_gap	ppn	max_level	baseline	unit_amp	cols
Phenotype_A	Real	73316	FALSE	10000	3561	636	0.5825	1	0.1817	8.6333	inferred: chan=col2 cur=col1 time=none, assuming 10000 Hz
Phenotype_A	Diffusion	100000	TRUE	10000	1533	582	0.5887	1	0.3384	8.2958	named columns: chan=col2 cur=col3 time=col1
Phenotype_A	GAN	97400	FALSE	10000	2459	426	0.5262	1	-0.0506	8.9217	inferred: chan=col2 cur=col1 time=none, assuming 10000 Hz
Phenotype_B	Real	100044	FALSE	10000	11617	1191	0.0889	1	0.0073	9.9585	inferred: chan=col2 cur=col1 time=none, assuming 10000 Hz
Phenotype_B	Diffusion	100000	TRUE	10000	9668	520	0.0790	1	0.0337	9.7086	named columns: chan=col2 cur=col3 time=col1
Phenotype_B	GAN	490896	FALSE	10000	6922	1262	0.1540	1	-0.1738	10.0189	inferred: chan=col2 cur=col1 time=none, assuming 10000 Hz
Phenotype_C	Real	150752	FALSE	10000	7709	8677	0.6538	1	-0.4125	0.4922	inferred: chan=col2 cur=col1 time=none, assuming 10000 Hz
Phenotype_C	Diffusion	100000	TRUE	10000	6570	3540	0.7548	1	-0.4164	0.4792	named columns: chan=col2 cur=col3 time=col1
Phenotype_C	GAN	779200	FALSE	10000	5872	1881	0.4855	1	-0.3736	0.4168	inferred: chan=col2 cur=col1 time=none, assuming 10000 Hz
Phenotype_D	Real	255840	TRUE	10000	11	1912	0.9879	1	0.3400	0.4260	inferred: chan=col3 cur=col2 time=col1
Phenotype_D	Diffusion	100000	TRUE	10000	17	1937	0.9879	1	0.3654	0.3949	named columns: chan=col2 cur=col3 time=col1

phenotype	arm	n	has_time	fs	longest_closed	longest_open	max_level	baseline	unit_amps	notes
Phenotype_A	GAN	173336	TRUE	5000	5301	1499	0.8754	1	1.8482	0.2597	inferred: chan=col3 cur=col2 time=col1
Phenotype_B	Real	366588	TRUE	20000	9768	862	0.8113	1	4.0960	7.6925	inferred: chan=col3 cur=col2 time=col1
Phenotype_C	Diffusion	100000	TRUE	10000	284	616	0.8571	1	4.9219	6.8038	named columns: chan=col2 cur=col3 time=col1
Phenotype_D	GAN	278528	FALSE	10000	149	694	0.8863	1	5.0396	6.4220	inferred: chan=col2 cur=col1 time=none, assuming 10000 Hz

```
readr::write_csv(inventory, file.path(CFG$out_dir, "preflight_inventory.csv"))
```

Sample rate

Only the seed record's time column reflects a real acquisition. `diffusion_app.py` writes its export time as `arange(n) * d * SAMPLE_DT` with `SAMPLE_DT` fixed at 1e-4, so a DDPM record always claims 10 kHz whatever it was trained on. A GAN record's time column is whatever DeepGANnel wrote, and when that disagrees with the seed there are two readings, neither of which can be settled from the file alone:

- **equivalent** (`rate_policy = "seed"`): the time column is cosmetic. The record is a sample-for-sample stand-in for the seed and inherits its rate.
- **honest** (`rate_policy = "own"`): the record really was produced at that rate, so it has half the bandwidth and its events last twice as long in wall-clock terms.

Relabelling a genuinely decimated record makes its dwell times and spectrum wrong; taking a cosmetic label at face value does the same in the other direction. So both are computed and compared below.

```
seed_fs <- inventory |>
  dplyr::filter(arm == "Real") |>
  dplyr::mutate(fs = dplyr::if_else(has_time, fs, CFG$sample_hz))
seed_fs <- setNames(seed_fs$fs, seed_fs$phenotype)

rates_under <- function(policy) {
  inventory |>
    dplyr::transmute(
      phenotype, arm,
      fs = if (identical(policy, "own")) {
        dplyr::if_else(has_time, fs, unname(seed_fs[phenotype]))
      } else {
        unname(seed_fs[phenotype])
      })
}

rate_map <- function(policy) {
  r <- rates_under(policy)
  split(setNames(r$fs, r$arm), r$phenotype)
}

disputed <- rates_under("own") |>
```

```
  dplyr::rename(fs_own = fs) |>
  dplyr::left_join(dplyr::rename(rates_under("seed"), fs_seed = fs),
                   by = c("phenotype", "arm")) |>
  dplyr::filter(abs(fs_own - fs_seed) / fs_seed > 0.01)

knitr::kable(rates_under("seed") |>
               tidyr::pivot_wider(names_from = arm, values_from = fs),
             digits = 1, caption = "Rate under the equivalent policy (Hz).")
```

Table 3: Rate under the equivalent policy (Hz).

phenotype	Real	Diffusion	GAN
Phenotype_A	10000	10000	10000
Phenotype_B	10000	10000	10000
Phenotype_C	10000	10000	10000
Phenotype_D	10000	10000	10000
Phenotype_E	20000	20000	20000

```
if (nrow(disputed)) {
  knitr::kable(disputed, digits = 1,
               caption = "Arms where the two policies disagree. These are the only ones whose rate-depe
} else {
  cat("No arm's own time column disagrees with its seed; the two policies coincide.\n")
}
```

Table 4: Arms where the two policies disagree. These are the only ones whose rate-dependent metrics change between policies.

phenotype	arm	fs_own	fs_seed
Phenotype_D	GAN	5000	10000
Phenotype_E	Diffusion	10000	20000

```
FS_MAIN <- rate_map(CFG$rate_policy)
FS_ALT  <- rate_map(if (identical(CFG$rate_policy, "seed")) "own" else "seed")
```

The lag-1 autocorrelation of the single-level noise, reported after the analysis, is the physical evidence: decimation decorrelates adjacent samples, so a record genuinely at half the rate sits well below its own seed while unaffected phenotypes match.

Noise level and segment length for the spectral metrics

The noise metrics need contiguous samples at one conductance level. Which level that can be depends on the record: a quiet channel offers long closed periods, a record at Popen 0.99 offers long open ones and closures only a sample or two wide. The level is therefore chosen per phenotype as whichever *every* arm of that phenotype can support, and reported so it can be stated in the legend.

Reporting an open-level spectrum is not a fudge: the requirement is that no gating transition falls inside a segment, which the open level satisfies exactly as well. It does measure something slightly different — open-channel noise sits on top of the instrumentation noise — but it is the same quantity in both arms of the comparison, which is all the paired metric needs.

```r
cap_at <- inventory |>
  dplyr::group_by(phenotype) |>
  dplyr::summarise(closed = min(longest_closed), open = min(longest_open),
                   .groups = "drop")

levels_used <- cap_at |>
  dplyr::mutate(
    level = if (identical(CFG$noise_level, "auto")) {
      dplyr::if_else(open > closed, "open", "closed")
    } else {
      CFG$noise_level
    },
    cap = dplyr::if_else(level == "open", open, closed))

if (any(levels_used$cap == 0)) {
  z <- levels_used |> dplyr::filter(cap == 0)
  stop("No usable ", paste(unique(z$level), collapse = "/"), " segments at all in: ",
       paste(z$phenotype, collapse = ", "),
       ". Set CFG$noise_level to the other level, or exclude these phenotypes.")
}

knitr::kable(levels_used, digits = 0,
             caption = "Longest run every arm can supply at each level, and the level used. 'cap' is the
```

Table 5: Longest run every arm can supply at each level, and the level used. 'cap' is the binding constraint on segment length.

phenotype	closed	open	level	cap
Phenotype_A	1533	426	closed	1533
Phenotype_B	6922	520	closed	6922
Phenotype_C	5872	1881	closed	5872
Phenotype_D	11	1499	open	1499
Phenotype_E	149	616	open	616

```r
cap <- min(levels_used$cap)
limiting <- levels_used |> dplyr::slice_min(cap, n = 1, with_ties = FALSE)

NPERSEG <- if (identical(CFG$psd_seg, "auto")) 2^floor(log2(cap)) else as.integer(CFG$psd_seg)

if (NPERSEG > cap) {
  stop("CFG$psd_seg = ", NPERSEG, " but ", limiting$phenotype,
       " supports only ", cap, " samples at its ", limiting$level,
       " level. Set CFG$psd_seg = \"auto\" or reduce it to at most ",
       2^floor(log2(cap)), ".")
}
if (NPERSEG < CFG$psd_seg_min) {
  stop(limiting$phenotype, " supports only ", cap, " contiguous samples at ",
       "either level (closed ", limiting$closed, ", open ", limiting$open,
       "), allowing ", NPERSEG, " samples per segment, below CFG$psd_seg_min = ",
       CFG$psd_seg_min, ". A spectrum that coarse is not worth reporting. ",
       "Exclude that phenotype from the spectral metrics with CFG$only, or ",
       "lower psd_seg_min deliberately and say so in the methods.")
```

```r
}
if (NPERSEG <= CFG$acf_lag_max) {
  stop("Segment length ", NPERSEG, " is not longer than CFG$acf_lag_max = ",
       CFG$acf_lag_max, ", so the ACF would be estimated over runs too short ",
       "to support it. Lower acf_lag_max.")
}

LEVEL_OF <- setNames(levels_used$level, levels_used$phenotype)

cat(sprintf("Segment length %d samples, set by %s at its %s level (cap %d).\n",
            NPERSEG, limiting$phenotype, limiting$level, cap))
```

```
## Segment length 512 samples, set by Phenotype_E at its open level (cap 616).
```

```r
for (r in sort(unique(unlist(FS_MAIN)))) {
  cat(sprintf("  at %g Hz: %.1f ms, %.1f Hz resolution\n", r,
              1000 * NPERSEG / r, r / NPERSEG))
}
```

```
##   at 10000 Hz: 51.2 ms, 19.5 Hz resolution
##   at 10000 Hz: 51.2 ms, 19.5 Hz resolution
##   at 20000 Hz: 25.6 ms, 39.1 Hz resolution
```

```r
if (dplyr::n_distinct(levels_used$level) > 1) {
  cat("\nLevels differ between phenotypes, so the noise metrics are paired\n",
      "within a phenotype but not comparable in absolute terms across them.\n",
      "State the level per panel in the figure legend.\n", sep = "")
}
```

```
##
## Levels differ between phenotypes, so the noise metrics are paired
## within a phenotype but not comparable in absolute terms across them.
## State the level per panel in the figure legend.
```

3. Per-phenotype analysis

Each phenotype is reduced to a long table of metrics, plus the curves needed for the pooled figures. ‘direction‘ records which way is better, because pooling a distance with a summary statistic that should *match* the real record would be meaningless otherwise.

```r
analyse_one <- function(row, arms_all, fs, level) {
  ph <- row$phenotype

  real    <- arms_all$Real
  arms    <- arms_all[setdiff(names(arms_all), "Real")]
  has_gan <- "GAN" %in% names(arms)
```

```
# ---- windows and MMD ------------------------------------------------
W <- list(Real = make_windows(real$current, CFG$seq_len, CFG$n_windows_mmd))
for (a in names(arms)) W[[a]] <- make_windows(arms[[a]]$current, CFG$seq_len,
                                              CFG$n_windows_mmd)
half <- floor(nrow(real) / 2)
W_r1 <- make_windows(real$current[1:half], CFG$seq_len, CFG$n_windows_mmd)
W_r2 <- make_windows(real$current[(half + 1):nrow(real)], CFG$seq_len, CFG$n_windows_mmd)
W_wn <- make_windows(rnorm(nrow(real), mean(real$current), sd(real$current)),
                     CFG$seq_len, CFG$n_windows_mmd)

# One bandwidth for every comparison in this phenotype, or the numbers are
# not on a single scale.
gam <- median_gamma(W$Real, W$Diffusion)

# ---- single-level noise ---------------------------------------------
# Runs are kept separate and every record has already been shown to hold at
# least one of NPERSEG samples, so there is no missing-data branch here.
runs <- lapply(arms_all, level_runs, level = level, min_len = NPERSEG)

psd <- purrr::imap_dfr(runs, function(x, a)
  dplyr::mutate(welch_psd(x, fs[[a]], NPERSEG), source = a))
acf_tbl <- purrr::imap_dfr(runs, function(x, a)
  dplyr::mutate(acf_of(x, CFG$acf_lag_max, fs[[a]]), source = a))

# Lag-1 autocorrelation of the single-level noise, per arm. A record genuinely
# at a lower rate than the one assumed for it sits well below its own seed.
lag1 <- acf_tbl |> dplyr::filter(lag_n == 2)
lag1 <- setNames(lag1$acf, lag1$source)

# Two arms at different rates have different frequency and lag grids, so both
# comparisons interpolate onto a shared axis rather than pivoting on the grid
# value. Under the equivalent policy the grids already coincide and the
# interpolation is a no-op.
curve <- function(tbl, a, xvar, yvar) {
  d <- tbl[tbl$source == a, ]
  list(x = d[[xvar]], y = d[[yvar]])
}

psd_rms <- function(a) {
  r <- curve(psd, "Real", "freq", "psd")
  g <- curve(psd, a, "freq", "psd")
  hi <- min(CFG$filter_hz, max(r$x), max(g$x))
  lo <- max(min(r$x), min(g$x))
  grid <- exp(seq(log(lo), log(hi), length.out = 512))
  rms_diff(approx(r$x, log10(r$y), grid)$y, approx(g$x, log10(g$y), grid)$y)
}

acf_rms <- function(a) {
  r <- curve(acf_tbl, "Real", "lag_ms", "acf")
  g <- curve(acf_tbl, a, "lag_ms", "acf")
  grid <- seq(0, min(max(r$x), max(g$x)), length.out = CFG$acf_lag_max + 1)
  rms_diff(approx(r$x, r$y, grid)$y, approx(g$x, g$y, grid)$y)
}
```

```
# ---- slow envelope ----------------------------------------------------
env <- list(Real = block_mean(real$current, CFG$slow_factor))
for (a in names(arms)) env[[a]] <- block_mean(arms[[a]]$current, CFG$slow_factor)
We <- lapply(env, function(e) make_windows(e, min(64, length(e)), CFG$n_windows_mmd))

# ---- dwell times ------------------------------------------------------
dw <- purrr::imap_dfr(arms_all, function(d, a)
  dplyr::mutate(dwell_times(d$channels, fs[[a]]), source = a))
mean_dwell <- function(a, s) {
  v <- dw |> dplyr::filter(source == a, state == s) |> dplyr::pull(dwell_ms)
  if (!length(v)) stop(ph, "/", a, ": no ", s, " dwells to average")
  mean(v)
}

level_pts <- function(d) d$current[at_level(d, level)]

# ---- assemble ---------------------------------------------------------
# One metric name whatever the level, so the pooled comparison keeps all
# phenotypes; the level itself is recorded in the provenance table.
sd_name <- "Noise SD (pA)"
M <- function(metric, method, direction, value)
  tibble::tibble(phenotype = ph, metric = metric, method = method,
                 direction = direction, value = as.numeric(value))

rows <- dplyr::bind_rows(
  M("MMD vs real",          "Real floor",  "reference", mmd_rbf(W_r1, W_r2, gamma = gam)),
  M("MMD vs real",          "White noise", "reference", mmd_rbf(W$Real, W_wn, gamma = gam)),
  M(sd_name,                "Real",        "reference", sd(level_pts(real))),
  M("Envelope SD (pA)",     "Real",        "reference", sd(env$Real)),
  M("Open probability",     "Real",        "reference", mean(real$channels > 0)),
  M("Mean open dwell (ms)", "Real",        "reference", mean_dwell("Real", "Open")),
  M("Mean closed dwell (ms)", "Real",      "reference", mean_dwell("Real", "Closed")),
  purrr::map_dfr(names(arms), function(a) {
    pm <- mmd_permutation_p(W$Real, W[[a]], gamma = gam)
    dplyr::bind_rows(
      M("MMD vs real",            a, "lower",  mmd_rbf(W$Real, W[[a]], gamma = gam)),
      M("MMD permutation p",      a, "higher", pm$p),
      M("KS amplitude",           a, "lower",
        unname(ks_safe(real$current, arms[[a]]$current)$statistic)),
      M("log10 PSD RMS",          a, "lower",  psd_rms(a)),
      M("ACF RMS",                a, "lower",  acf_rms(a)),
      M("MMD envelope",           a, "lower",  mmd_rbf(We$Real, We[[a]])),
      M(sd_name,                  a, "match_real", sd(level_pts(arms[[a]]))),
      M("Envelope SD (pA)",       a, "match_real", sd(env[[a]])),
      M("Open probability",       a, "match_real", mean(arms[[a]]$channels > 0)),
      M("Mean open dwell (ms)",   a, "match_real", mean_dwell(a, "Open")),
      M("Mean closed dwell (ms)", a, "match_real", mean_dwell(a, "Closed"))
    )
  })
)

# ---- curves and provenance --------------------------------------------
# time_ms is built from the sample index and the rate assigned to the arm, not
# from the file's own time column, which for a DDPM export is a fixed 1e-4
```

```
  # step regardless of what it was trained on and makes panels differ in width.
  traces <- purrr::imap_dfr(arms_all, function(d, a) {
    n <- min(nrow(d), round(CFG$trace_secs * fs[[a]]))
    s <- best_window(d$channels, n)
    d[s:(s + n - 1), ] |>
      dplyr::mutate(source = a, phenotype = ph,
                    time_ms = 1000 * (dplyr::row_number() - 1) / fs[[a]],
                    win_start = s, win_popen = mean(channels > 0))
  })

  amp <- purrr::imap_dfr(arms_all, function(d, a)
    tibble::tibble(current = d$current, source = a, phenotype = ph))

  info <- tibble::tibble(
    phenotype = ph,
    seed_file = basename(row$raw),
    gan_file  = if (has_gan) basename(row$gan) else "none",
    ddpm_file = basename(row$ddpm),
    n_real = nrow(real),
    n_ddpm = nrow(arms$Diffusion),
    n_gan  = if (has_gan) nrow(arms$GAN) else NA_integer_,
    popen_real = mean(real$channels > 0),
    max_level_real = max(real$channels),
    fs_real = fs[["Real"]],
    fs_ddpm = fs[["Diffusion"]],
    fs_gan  = if (has_gan) fs[["GAN"]] else NA_real_,
    noise_level = level,
    psd_nperseg = NPERSEG,
    psd_seg_ms = 1000 * NPERSEG / fs[["Real"]],
    psd_n_seg_real = psd$n_seg[1],
    lag1_real = unname(lag1["Real"]),
    lag1_ddpm = unname(lag1["Diffusion"]),
    lag1_gan  = if (has_gan) unname(lag1["GAN"]) else NA_real_,
    gamma = gam
  )

  list(metrics = rows,
       psd = dplyr::mutate(psd, phenotype = ph),
       acf = dplyr::mutate(acf_tbl, phenotype = ph),
       dwell = dplyr::mutate(dw, phenotype = ph),
       env = purrr::imap_dfr(env, function(e, a)
         tibble::tibble(idx = seq_along(e), value = e, source = a, phenotype = ph)),
       traces = traces,
       amp = amp,
       info = info)
}

# The cache is keyed on the segment length, the rates in force and the policy
# label: results computed under different assumptions are not comparable with
# these and must not be silently reused.
run_one <- function(row, fs, policy) {
  key <- sprintf("%s_seg%d_%s_%s_%s", row$phenotype, NPERSEG,
                 LEVEL_OF[[row$phenotype]], policy,
                 paste(sprintf("%s%g", substr(names(fs), 1, 1), fs), collapse = ""))
```

```
  cache_file <- file.path(CFG$out_dir, "cache", paste0(key, ".rds"))
  if (CFG$cache && file.exists(cache_file)) {
    message("Reusing cached ", key)
    return(readRDS(cache_file))
  }
  message("Analysing ", row$phenotype, " (", policy, " rates)")
  res <- analyse_one(row, recs[[row$phenotype]], fs, LEVEL_OF[[row$phenotype]])
  if (CFG$cache) saveRDS(res, cache_file)
  res
}

results <- purrr::map(seq_len(nrow(manifest)), function(i) {
  row <- manifest[i, ]
  run_one(row, FS_MAIN[[row$phenotype]], CFG$rate_policy)
})
names(results) <- manifest$phenotype

pull_all <- function(what) purrr::map_dfr(results, what)

metrics <- pull_all("metrics")
info    <- pull_all("info")

# Nothing missing gets past this point. An NA here would become a phenotype
# silently dropped from a paired comparison, which is worse than a failed knit.
bad <- dplyr::filter(metrics, !is.finite(value))
if (nrow(bad)) {
  stop("Non-finite metrics, which must be understood rather than dropped:\n",
       paste(sprintf("  %s / %s / %s", bad$phenotype, bad$method, bad$metric),
             collapse = "\n"))
}

knitr::kable(
  info |> dplyr::select(phenotype, seed_file, ddpm_file, gan_file,
                        n_real, n_ddpm, n_gan, popen_real, max_level_real),
  digits = 4, caption = "Table 1. Records analysed."
)
```

Table 6: Table 1. Records analysed.

phenotype	seed_file	ddpm_file	gan_file	n_real	n_ddpm	n_gan	popen_real	max_level_real
Phenotype_A_DDPM_PYR_GAN	4096PYRraw.csv	Phenotype_A_DDPM_PYR_GAN.csv	4096PYRGAN.csv	73316	1e+05	97400	0.5825	1
Phenotype_B_DDPM_pad01_GAN	4096pad01raw.csv	Phenotype_B_DDPM_pad01_GAN.csv	4096pad01GAN.csv	1000445	1e+05	490896	0.0889	1
Phenotype_C_DDPM_lina2_GAN	4096lina11raw.csv	Phenotype_C_DDPM_lina2_GAN_data.csv	4096lina2GAN_data.csv	150752	1e+05	779200	0.6538	1
Phenotype_D_DDPM_pvt_GAN	pvtraw.csv	Phenotype_D_DDPM_pvt_GAN.csv	pvtGAN.csv	255840	1e+05	173336	0.9879	1
Phenotype_E_DDPM_c035_GAN	c035[NF=50Hz]0-32_4kraw.csv	Phenotype_E_DDPM_c035_GAN.csv	c035[NF=50Hz]0-32_4kGAN.csv	366588	1e+05	278528	0.8113	1

```
knitr::kable(info |> dplyr::select(phenotype, noise_level, fs_real, fs_ddpm,
                                   fs_gan, psd_nperseg, psd_seg_ms,
                                   psd_n_seg_real, gamma),
             digits = 4,
             caption = "Rate applied, spectral segment length, segments contributing to the real spectru
```

Table 7: Rate applied, spectral segment length, segments contributing to the real spectrum, and kernel bandwidth.

phenotype	noise_level	fs_real	fs_ddpm	fs_gan	psd_nperseg	psd_seg_ms	psd_n_seg_real	gamma
Phenotype_A	closed	10000	10000	10000	512	51.2	36	0.0003
Phenotype_B	closed	10000	10000	10000	512	51.2	2934	0.0036
Phenotype_C	closed	10000	10000	10000	512	51.2	153	0.2175
Phenotype_D	open	10000	10000	10000	512	51.2	284	0.3241
Phenotype_E	open	20000	20000	20000	512	25.6	19	0.0010

```
knitr::kable(info |> dplyr::select(phenotype, lag1_real, lag1_ddpm, lag1_gan),
             digits = 4,
             caption = "Lag-1 autocorrelation of the single-level noise. An arm well below its own real
```

Table 8: Lag-1 autocorrelation of the single-level noise. An arm well below its own real value is genuinely at a lower rate than the one assumed for it, not merely mislabelled.

phenotype	lag1_real	lag1_ddpm	lag1_gan
Phenotype_A	0.6772	0.7432	0.4275
Phenotype_B	0.1758	0.1810	0.2438
Phenotype_C	0.1268	-0.0031	0.4135
Phenotype_D	0.8887	0.8749	0.6465
Phenotype_E	0.8570	0.8292	0.8269

```
readr::write_csv(info, file.path(CFG$out_dir, "provenance.csv"))
readr::write_csv(metrics, file.path(CFG$out_dir, "metrics_long.csv"))
```

Equivalent versus honest

Only rate-dependent metrics can move between the policies. The amplitude metrics, MMD and open probability are computed per sample and are identical either way, which is itself useful: it means the choice cannot be dodged by appealing to the metrics that happen to be convenient.

```
alt_phenos <- unique(disputed$phenotype)

if (!length(alt_phenos)) {
  cat("No arm's rate is disputed, so the two policies give identical results.\n")
} else {
  alt <- purrr::map(alt_phenos, function(ph) {
    row <- manifest[manifest$phenotype == ph, ]
    run_one(row, FS_ALT[[ph]], if (identical(CFG$rate_policy, "seed")) "own" else "seed")
  })
  names(alt) <- alt_phenos

  rate_sensitive <- c("log10 PSD RMS", "ACF RMS",
                      "Mean open dwell (ms)", "Mean closed dwell (ms)")

  cmp <- dplyr::bind_rows(
```

```
    metrics |> dplyr::filter(phenotype %in% alt_phenos) |>
      dplyr::mutate(policy = CFG$rate_policy),
    purrr::map_dfr(alt, "metrics") |>
      dplyr::mutate(policy = if (identical(CFG$rate_policy, "seed")) "own" else "seed")
  ) |>
    dplyr::filter(metric %in% rate_sensitive) |>
    dplyr::mutate(policy = dplyr::recode(policy, seed = "equivalent", own = "honest")) |>
    dplyr::select(phenotype, metric, method, policy, value) |>
    tidyr::pivot_wider(names_from = policy, values_from = value) |>
    dplyr::mutate(ratio = honest / equivalent)

  knitr::kable(cmp, digits = 4,
               caption = "Rate-dependent metrics under both readings. Everything else is unchanged by c
  readr::write_csv(cmp, file.path(CFG$out_dir, "rate_policy_comparison.csv"))

  lag_cmp <- dplyr::bind_rows(
    info |> dplyr::filter(phenotype %in% alt_phenos) |>
      dplyr::mutate(policy = CFG$rate_policy),
    purrr::map_dfr(alt, "info") |>
      dplyr::mutate(policy = if (identical(CFG$rate_policy, "seed")) "own" else "seed")
  ) |>
    dplyr::select(phenotype, policy, lag1_real, lag1_ddpm, lag1_gan)
  knitr::kable(lag_cmp, digits = 4,
               caption = "Lag-1 autocorrelation, which does not depend on the rate label at all and is
}
```

Table 9: Lag-1 autocorrelation, which does not depend on the rate label at all and is therefore the evidence.

phenotype	policy	lag1_real	lag1_ddpm	lag1_gan
Phenotype_D	seed	0.8887	0.8749	0.6465
Phenotype_E	seed	0.8570	0.8292	0.8269
Phenotype_D	own	0.8887	0.8749	0.6465
Phenotype_E	own	0.8570	0.8292	0.8269

How to read this

The lag-1 table is the part that decides it, because it is computed per sample and is untouched by whichever label is applied. If the disputed arm's value sits close to its own seed record, the samples are correlated like the seed's are and the record is a sample-for-sample stand-in with a cosmetic time column: **equivalent** is right, and the honest reading inflates its dwell times by the rate ratio for no physical reason. If it sits well below, adjacent samples really are less correlated, the record was produced at the lower rate, and **honest** is right — but then the record has less bandwidth than the seed, which is a genuine limitation of that arm and belongs in the results rather than being corrected away.

The dwell ratios are the sanity check on whichever answer you accept: under the honest reading a 5 kHz record's mean dwells double exactly, so if the honest dwell times land implausibly far from the seed's while the equivalent ones sit close, that is evidence for the equivalent reading and vice versa.

4. Scoring

Distances should be small, the permutation p should be large, and summary statistics such as the noise SD should *match* the real record rather than approach zero. The score column puts all three on a "smaller is better" footing except the p, which is flagged separately.

```
refs <- metrics |>
  dplyr::filter(direction == "reference", method == "Real") |>
  dplyr::select(phenotype, metric, ref = value)

scored <- metrics |>
  dplyr::filter(direction %in% c("lower", "higher", "match_real")) |>
  dplyr::left_join(refs, by = c("phenotype", "metric")) |>
  dplyr::mutate(
    score  = dplyr::if_else(direction == "match_real", abs(value - ref), value),
    better = dplyr::if_else(direction == "higher", "higher", "lower")
  ) |>
  dplyr::filter(is.finite(score))

knitr::kable(
  scored |> dplyr::arrange(metric, method, phenotype) |>
    dplyr::select(metric, method, phenotype, value, ref, score),
  digits = 5, caption = "Table 2. Raw values and the comparable score derived from them."
)
```

Table 10: Table 2. Raw values and the comparable score derived from them.

metric	method	phenotype	value	ref	score
ACF RMS	Diffusion	Phenotype_A	0.07455	NA	0.07455
ACF RMS	Diffusion	Phenotype_B	0.00443	NA	0.00443
ACF RMS	Diffusion	Phenotype_C	0.03188	NA	0.03188
ACF RMS	Diffusion	Phenotype_D	0.03392	NA	0.03392
ACF RMS	Diffusion	Phenotype_E	0.04057	NA	0.04057
ACF RMS	GAN	Phenotype_A	0.13948	NA	0.13948
ACF RMS	GAN	Phenotype_B	0.01416	NA	0.01416
ACF RMS	GAN	Phenotype_C	0.14467	NA	0.14467
ACF RMS	GAN	Phenotype_D	0.04745	NA	0.04745
ACF RMS	GAN	Phenotype_E	0.04466	NA	0.04466
Envelope SD (pA)	Diffusion	Phenotype_A	3.37378	3.63859	0.26482
Envelope SD (pA)	Diffusion	Phenotype_B	2.41091	2.68539	0.27448
Envelope SD (pA)	Diffusion	Phenotype_C	0.19794	0.23853	0.04059
Envelope SD (pA)	Diffusion	Phenotype_D	0.04299	0.05182	0.00883
Envelope SD (pA)	Diffusion	Phenotype_E	1.72760	2.41432	0.68672
Envelope SD (pA)	GAN	Phenotype_A	3.99808	3.63859	0.35949
Envelope SD (pA)	GAN	Phenotype_B	3.42400	2.68539	0.73861
Envelope SD (pA)	GAN	Phenotype_C	0.20491	0.23853	0.03362
Envelope SD (pA)	GAN	Phenotype_D	0.52472	0.05182	0.47290
Envelope SD (pA)	GAN	Phenotype_E	1.46457	2.41432	0.94975
KS amplitude	Diffusion	Phenotype_A	0.06500	NA	0.06500
KS amplitude	Diffusion	Phenotype_B	0.01639	NA	0.01639
KS amplitude	Diffusion	Phenotype_C	0.16002	NA	0.16002

metric	method	phenotype	value	ref	score
KS amplitude	Diffusion	Phenotype_D	0.05503	NA	0.05503
KS amplitude	Diffusion	Phenotype_E	0.07393	NA	0.07393
KS amplitude	GAN	Phenotype_A	0.09922	NA	0.09922
KS amplitude	GAN	Phenotype_B	0.09704	NA	0.09704
KS amplitude	GAN	Phenotype_C	0.19508	NA	0.19508
KS amplitude	GAN	Phenotype_D	0.88856	NA	0.88856
KS amplitude	GAN	Phenotype_E	0.11778	NA	0.11778
MMD envelope	Diffusion	Phenotype_A	0.00992	NA	0.00992
MMD envelope	Diffusion	Phenotype_B	0.00138	NA	0.00138
MMD envelope	Diffusion	Phenotype_C	0.03522	NA	0.03522
MMD envelope	Diffusion	Phenotype_D	0.01644	NA	0.01644
MMD envelope	Diffusion	Phenotype_E	0.00809	NA	0.00809
MMD envelope	GAN	Phenotype_A	0.02699	NA	0.02699
MMD envelope	GAN	Phenotype_B	0.03671	NA	0.03671
MMD envelope	GAN	Phenotype_C	0.11529	NA	0.11529
MMD envelope	GAN	Phenotype_D	0.97779	NA	0.97779
MMD envelope	GAN	Phenotype_E	0.02497	NA	0.02497
MMD permutation p	Diffusion	Phenotype_A	0.32338	NA	0.32338
MMD permutation p	Diffusion	Phenotype_B	0.21891	NA	0.21891
MMD permutation p	Diffusion	Phenotype_C	0.00498	NA	0.00498
MMD permutation p	Diffusion	Phenotype_D	0.00498	NA	0.00498
MMD permutation p	Diffusion	Phenotype_E	0.00498	NA	0.00498
MMD permutation p	GAN	Phenotype_A	0.00995	NA	0.00995
MMD permutation p	GAN	Phenotype_B	0.00995	NA	0.00995
MMD permutation p	GAN	Phenotype_C	0.00498	NA	0.00498
MMD permutation p	GAN	Phenotype_D	0.00498	NA	0.00498
MMD permutation p	GAN	Phenotype_E	0.00498	NA	0.00498
MMD vs real	Diffusion	Phenotype_A	0.00019	NA	0.00019
MMD vs real	Diffusion	Phenotype_B	0.00018	NA	0.00018
MMD vs real	Diffusion	Phenotype_C	0.03907	NA	0.03907
MMD vs real	Diffusion	Phenotype_D	0.00719	NA	0.00719
MMD vs real	Diffusion	Phenotype_E	0.00629	NA	0.00629
MMD vs real	GAN	Phenotype_A	0.01028	NA	0.01028
MMD vs real	GAN	Phenotype_B	0.00270	NA	0.00270
MMD vs real	GAN	Phenotype_C	0.04690	NA	0.04690
MMD vs real	GAN	Phenotype_D	0.33838	NA	0.33838
MMD vs real	GAN	Phenotype_E	0.00695	NA	0.00695
Mean closed dwell (ms)	Diffusion	Phenotype_A	3.60149	3.40100	0.20049
Mean closed dwell (ms)	Diffusion	Phenotype_B	161.57895	195.17495	33.59600
Mean closed dwell (ms)	Diffusion	Phenotype_C	25.54062	37.27500	11.73438
Mean closed dwell (ms)	Diffusion	Phenotype_D	0.39318	0.38005	0.01313
Mean closed dwell (ms)	Diffusion	Phenotype_E	0.80298	1.19494	0.39196
Mean closed dwell (ms)	GAN	Phenotype_A	4.48494	3.40100	1.08394
Mean closed dwell (ms)	GAN	Phenotype_B	123.96925	195.17495	71.20569
Mean closed dwell (ms)	GAN	Phenotype_C	14.06786	37.27500	23.20714
Mean closed dwell (ms)	GAN	Phenotype_D	1.41355	0.38005	1.03350
Mean closed dwell (ms)	GAN	Phenotype_E	0.53377	1.19494	0.66117
Mean open dwell (ms)	Diffusion	Phenotype_A	5.15508	4.75050	0.40458
Mean open dwell (ms)	Diffusion	Phenotype_B	14.10714	19.09399	4.98685
Mean open dwell (ms)	Diffusion	Phenotype_C	78.62604	70.91151	7.71453
Mean open dwell (ms)	Diffusion	Phenotype_D	32.07435	30.89670	1.17765
Mean open dwell (ms)	Diffusion	Phenotype_E	4.81500	5.13823	0.32323

metric	method	phenotype	value	ref	score
Mean open dwell (ms)	GAN	Phenotype_A	4.97573	4.75050	0.22523
Mean open dwell (ms)	GAN	Phenotype_B	22.56687	19.09399	3.47287
Mean open dwell (ms)	GAN	Phenotype_C	13.27249	70.91151	57.63902
Mean open dwell (ms)	GAN	Phenotype_D	9.92394	30.89670	20.97276
Mean open dwell (ms)	GAN	Phenotype_E	4.16018	5.13823	0.97806
Noise SD (pA)	Diffusion	Phenotype_A	1.43623	1.43839	0.00216
Noise SD (pA)	Diffusion	Phenotype_B	1.21333	1.17360	0.03973
Noise SD (pA)	Diffusion	Phenotype_C	0.04066	0.04818	0.00752
Noise SD (pA)	Diffusion	Phenotype_D	0.09742	0.10899	0.01156
Noise SD (pA)	Diffusion	Phenotype_E	0.99899	1.02172	0.02273
Noise SD (pA)	GAN	Phenotype_A	1.55120	1.43839	0.11281
Noise SD (pA)	GAN	Phenotype_B	1.24561	1.17360	0.07201
Noise SD (pA)	GAN	Phenotype_C	0.05601	0.04818	0.00782
Noise SD (pA)	GAN	Phenotype_D	0.61861	0.10899	0.50962
Noise SD (pA)	GAN	Phenotype_E	1.11970	1.02172	0.09799
Open probability	Diffusion	Phenotype_A	0.58871	0.58251	0.00620
Open probability	Diffusion	Phenotype_B	0.07900	0.08894	0.00994
Open probability	Diffusion	Phenotype_C	0.75481	0.65384	0.10097
Open probability	Diffusion	Phenotype_D	0.98789	0.98786	0.00003
Open probability	Diffusion	Phenotype_E	0.85707	0.81127	0.04580
Open probability	GAN	Phenotype_A	0.52618	0.58251	0.05633
Open probability	GAN	Phenotype_B	0.15400	0.08894	0.06506
Open probability	GAN	Phenotype_C	0.48545	0.65384	0.16838
Open probability	GAN	Phenotype_D	0.87539	0.98786	0.11247
Open probability	GAN	Phenotype_E	0.88632	0.81127	0.07505
log10 PSD RMS	Diffusion	Phenotype_A	0.79061	NA	0.79061
log10 PSD RMS	Diffusion	Phenotype_B	0.10178	NA	0.10178
log10 PSD RMS	Diffusion	Phenotype_C	0.27340	NA	0.27340
log10 PSD RMS	Diffusion	Phenotype_D	0.20318	NA	0.20318
log10 PSD RMS	Diffusion	Phenotype_E	0.44855	NA	0.44855
log10 PSD RMS	GAN	Phenotype_A	0.55108	NA	0.55108
log10 PSD RMS	GAN	Phenotype_B	0.17175	NA	0.17175
log10 PSD RMS	GAN	Phenotype_C	0.66584	NA	0.66584
log10 PSD RMS	GAN	Phenotype_D	1.29468	NA	1.29468
log10 PSD RMS	GAN	Phenotype_E	0.19499	NA	0.19499

```
readr::write_csv(scored, file.path(CFG$out_dir, "scores_long.csv"))
```

Table 3: head to head, per phenotype

```
table3 <- scored |>
  dplyr::select(metric, phenotype, method, value) |>
  tidyr::pivot_wider(names_from = method, values_from = value) |>
  dplyr::left_join(refs, by = c("phenotype", "metric")) |>
  dplyr::rename(`Reference (real)` = ref) |>
  dplyr::arrange(metric, phenotype)

knitr::kable(table3, digits = 5,
             caption = paste("Table 3. Similarity metrics, computed identically",
```

```
                    "for both methods. Lower is better throughout except",
                    "the permutation p, where a large value means the",
                    "difference from real is no bigger than relabelling",
                    "produces."))
```

Table 11: Table 3. Similarity metrics, computed identically for both methods. Lower is better throughout except the permutation p, where a large value means the difference from real is no bigger than relabelling produces.

metric	phenotype	Diffusion	GAN	Reference (real)
ACF RMS	Phenotype_A	0.07455	0.13948	NA
ACF RMS	Phenotype_B	0.00443	0.01416	NA
ACF RMS	Phenotype_C	0.03188	0.14467	NA
ACF RMS	Phenotype_D	0.03392	0.04745	NA
ACF RMS	Phenotype_E	0.04057	0.04466	NA
Envelope SD (pA)	Phenotype_A	3.37378	3.99808	3.63859
Envelope SD (pA)	Phenotype_B	2.41091	3.42400	2.68539
Envelope SD (pA)	Phenotype_C	0.19794	0.20491	0.23853
Envelope SD (pA)	Phenotype_D	0.04299	0.52472	0.05182
Envelope SD (pA)	Phenotype_E	1.72760	1.46457	2.41432
KS amplitude	Phenotype_A	0.06500	0.09922	NA
KS amplitude	Phenotype_B	0.01639	0.09704	NA
KS amplitude	Phenotype_C	0.16002	0.19508	NA
KS amplitude	Phenotype_D	0.05503	0.88856	NA
KS amplitude	Phenotype_E	0.07393	0.11778	NA
MMD envelope	Phenotype_A	0.00992	0.02699	NA
MMD envelope	Phenotype_B	0.00138	0.03671	NA
MMD envelope	Phenotype_C	0.03522	0.11529	NA
MMD envelope	Phenotype_D	0.01644	0.97779	NA
MMD envelope	Phenotype_E	0.00809	0.02497	NA
MMD permutation p	Phenotype_A	0.32338	0.00995	NA
MMD permutation p	Phenotype_B	0.21891	0.00995	NA
MMD permutation p	Phenotype_C	0.00498	0.00498	NA
MMD permutation p	Phenotype_D	0.00498	0.00498	NA
MMD permutation p	Phenotype_E	0.00498	0.00498	NA
MMD vs real	Phenotype_A	0.00019	0.01028	NA
MMD vs real	Phenotype_B	0.00018	0.00270	NA
MMD vs real	Phenotype_C	0.03907	0.04690	NA
MMD vs real	Phenotype_D	0.00719	0.33838	NA
MMD vs real	Phenotype_E	0.00629	0.00695	NA
Mean closed dwell (ms)	Phenotype_A	3.60149	4.48494	3.40100
Mean closed dwell (ms)	Phenotype_B	161.57895	123.96925	195.17495
Mean closed dwell (ms)	Phenotype_C	25.54062	14.06786	37.27500
Mean closed dwell (ms)	Phenotype_D	0.39318	1.41355	0.38005
Mean closed dwell (ms)	Phenotype_E	0.80298	0.53377	1.19494
Mean open dwell (ms)	Phenotype_A	5.15508	4.97573	4.75050
Mean open dwell (ms)	Phenotype_B	14.10714	22.56687	19.09399
Mean open dwell (ms)	Phenotype_C	78.62604	13.27249	70.91151
Mean open dwell (ms)	Phenotype_D	32.07435	9.92394	30.89670
Mean open dwell (ms)	Phenotype_E	4.81500	4.16018	5.13823
Noise SD (pA)	Phenotype_A	1.43623	1.55120	1.43839

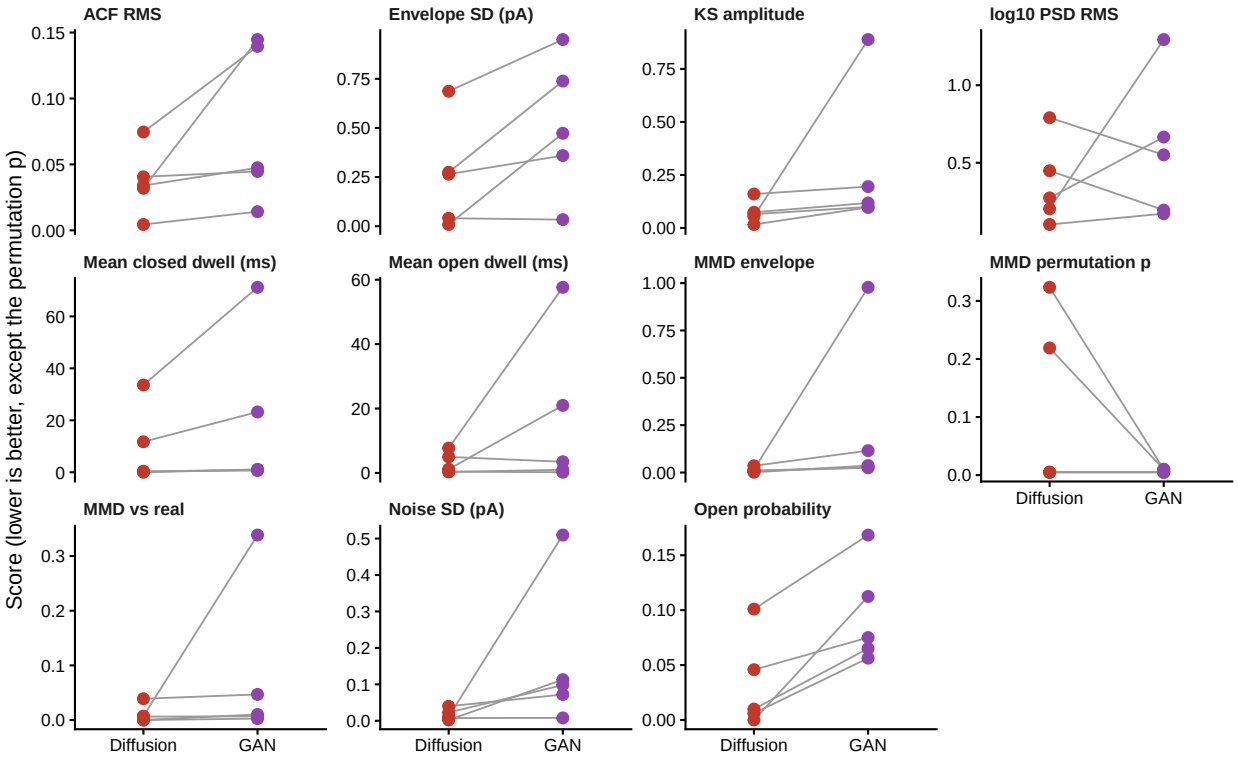
metric	phenotype	Diffusion	GAN	Reference (real)
Noise SD (pA)	Phenotype_B	1.21333	1.24561	1.17360
Noise SD (pA)	Phenotype_C	0.04066	0.05601	0.04818
Noise SD (pA)	Phenotype_D	0.09742	0.61861	0.10899
Noise SD (pA)	Phenotype_E	0.99899	1.11970	1.02172
Open probability	Phenotype_A	0.58871	0.52618	0.58251
Open probability	Phenotype_B	0.07900	0.15400	0.08894
Open probability	Phenotype_C	0.75481	0.48545	0.65384
Open probability	Phenotype_D	0.98789	0.87539	0.98786
Open probability	Phenotype_E	0.85707	0.88632	0.81127
log10 PSD RMS	Phenotype_A	0.79061	0.55108	NA
log10 PSD RMS	Phenotype_B	0.10178	0.17175	NA
log10 PSD RMS	Phenotype_C	0.27340	0.66584	NA
log10 PSD RMS	Phenotype_D	0.20318	1.29468	NA
log10 PSD RMS	Phenotype_E	0.44855	0.19499	NA

```
readr::write_csv(table3, file.path(CFG$out_dir, "table3_metrics.csv"))
```

5. Pooled comparison

The design is paired: both methods see the same seed record, so the quantity of interest is the within-phenotype difference. Between-phenotype variation in noise, open probability and duration is far larger than any method effect and would swamp it if the records were treated as independent samples.

```
pairable <- scored |>
  dplyr::filter(method %in% c("Diffusion", "GAN")) |>
  dplyr::group_by(metric, phenotype) |>
  dplyr::filter(dplyr::n_distinct(method) == 2) |>
  dplyr::ungroup()

n_ph <- dplyr::n_distinct(pairable$phenotype)

p_slope <- ggplot(pairable, aes(factor(method, levels = c("Diffusion", "GAN")),
                                score, group = phenotype)) +
  geom_line(colour = "grey60", linewidth = 0.4) +
  geom_point(aes(colour = method), size = 2.2) +
  facet_wrap(~ metric, scales = "free_y") +
  scale_colour_manual(values = pal, guide = "none") +
  labs(x = NULL, y = "Score (lower is better, except the permutation p)") +
  theme_channel()
p_slope
```




```
save_fig(p_slope, "fig_paired_metrics", width = 9, height = 6.5)
```

Each line is one phenotype. Lines that all slope the same way are the reportable result; lines that cross mean the ranking depends on which recording you happen to analyse.

```
wide <- pairable |>
  dplyr::select(metric, phenotype, better, method, score) |>
  tidyr::pivot_wider(names_from = method, values_from = score) |>
  dplyr::mutate(diff = Diffusion - GAN)

paired_stats <- wide |>
  dplyr::group_by(metric, better) |>
  dplyr::summarise(
    n              = dplyr::n(),
    median_diff    = median(diff),
    mean_diff      = mean(diff),
    sd_diff        = sd(diff),
    n_favour_diff  = sum(if (dplyr::first(better) == "lower") diff < 0 else diff > 0),
    p_paired_t     = if (dplyr::n() > 2 && sd(diff) > 0)
                       stats::t.test(diff)$p.value else NA_real_,
    ci_lo          = if (dplyr::n() > 2 && sd(diff) > 0)
                       stats::t.test(diff)$conf.int[1] else NA_real_,
    ci_hi          = if (dplyr::n() > 2 && sd(diff) > 0)
                       stats::t.test(diff)$conf.int[2] else NA_real_,
    p_sign         = stats::binom.test(sum(diff < 0), dplyr::n())$p.value,
    .groups = "drop"
  ) |>
  dplyr::mutate(verdict = dplyr::case_when(
```

```
    n_favour_diff == n ~ "consistent: Diffusion",
    n_favour_diff == 0 ~ "consistent: GAN",
    TRUE               ~ "inconsistent across phenotypes"
  ))

knitr::kable(paired_stats, digits = 5,
             caption = paste("Table 4. Pooled paired comparison, Diffusion minus",
                             "GAN within each phenotype. The t-test is on the",
                             "within-phenotype differences."))
```

Table 12: Table 4. Pooled paired comparison, Diffusion minus GAN within each phenotype. The t-test is on the within-phenotype differences.

metric	better	n	median_diff	mean_diff	sd_diff	n_favour_diff	p_paired	ci_lo	ci_hi	p_sign	verdict
ACF RMS	lower	5	-0.01354	-0.04102	0.04696	5	0.12254	-0.09933	0.01729	0.0625	consistent: Diffusion
Envelope SD (pA)	lower	5	-0.26303	-0.25579	0.21321	4	0.05508	-0.52053	0.00895	0.3750	inconsistent across phenotypes
KS amplitude	lower	5	-0.04385	-0.20547	0.35161	5	0.26139	-0.64205	0.23112	0.0625	consistent: Diffusion
MMD envelope	lower	5	-0.03533	-0.22214	0.41403	5	0.29645	-0.73623	0.29195	0.0625	consistent: Diffusion
MMD permutation p	higher	5	0.00000	0.10448	0.14775	2	0.18900	-0.07898	0.28794	0.0625	inconsistent across phenotypes
MMD vs real	lower	5	-0.00783	-0.07046	0.14580	5	0.34070	-0.25149	0.11058	0.0625	consistent: Diffusion
Mean closed dwell (ms)	lower	5	-1.02037	-10.25110	15.98892	5	0.22498	-30.10397	9.60177	0.0625	consistent: Diffusion
Mean open dwell (ms)	lower	5	-0.65482	-13.73622	22.04334	3	0.23594	-41.10664	13.63420	1.0000	inconsistent across phenotypes
Noise SD (pA)	lower	5	-0.07526	-0.14331	0.20268	5	0.18903	-0.39498	0.10836	0.0625	consistent: Diffusion
Open probability	lower	5	-0.05513	-0.06287	0.03095	5	0.01048	-0.10130	-0.02444	0.0625	consistent: Diffusion
log10 PSD RMS	lower	5	-0.06996	-0.21216	0.55832	3	0.44335	-0.90541	0.48108	1.0000	inconsistent across phenotypes

```
readr::write_csv(paired_stats, file.path(CFG$out_dir, "table4_paired_summary.csv"))
```

What this many phenotypes will and will not support

```
min_p_sign <- min(1, 2 * 0.5^n_ph)
cat(sprintf("Phenotypes pooled: %d\n", n_ph))
```

```
## Phenotypes pooled: 5
```

```
cat(sprintf("Smallest two-sided p a sign test can return here: %.4f\n", min_p_sign))
```

```
## Smallest two-sided p a sign test can return here: 0.0625
```

```
if (min_p_sign > 0.05) {
  cat("\nThat is above 0.05, so no sign test on these data can reach significance,\n",
      "however large or however unanimous the effect. The same floor applies to the\n",
      "Wilcoxon signed-rank test at this n. The p_sign column above is therefore\n",
      "reported for completeness and should not be leaned on.\n",
      "\nWhat these data do support: the per-phenotype values, the direction, whether\n",
      "it is unanimous, and the paired t interval on the differences. Those are the\n",
      "defensible summaries at this sample size.\n", sep = "")
} else {
  cat("\nA sign test is at least capable of reaching p < 0.05 here.\n")
}
```

```
##
## That is above 0.05, so no sign test on these data can reach significance,
## however large or however unanimous the effect. The same floor applies to the
## Wilcoxon signed-rank test at this n. The p_sign column above is therefore
## reported for completeness and should not be leaned on.
##
## What these data do support: the per-phenotype values, the direction, whether
## it is unanimous, and the paired t interval on the differences. Those are the
## defensible summaries at this sample size.
```

```
cat(sprintf("\nUnanimous metrics: %d of %d.\n",
            sum(paired_stats$verdict != "inconsistent across phenotypes"),
            nrow(paired_stats)))
```

```
##
## Unanimous metrics: 7 of 11.
```

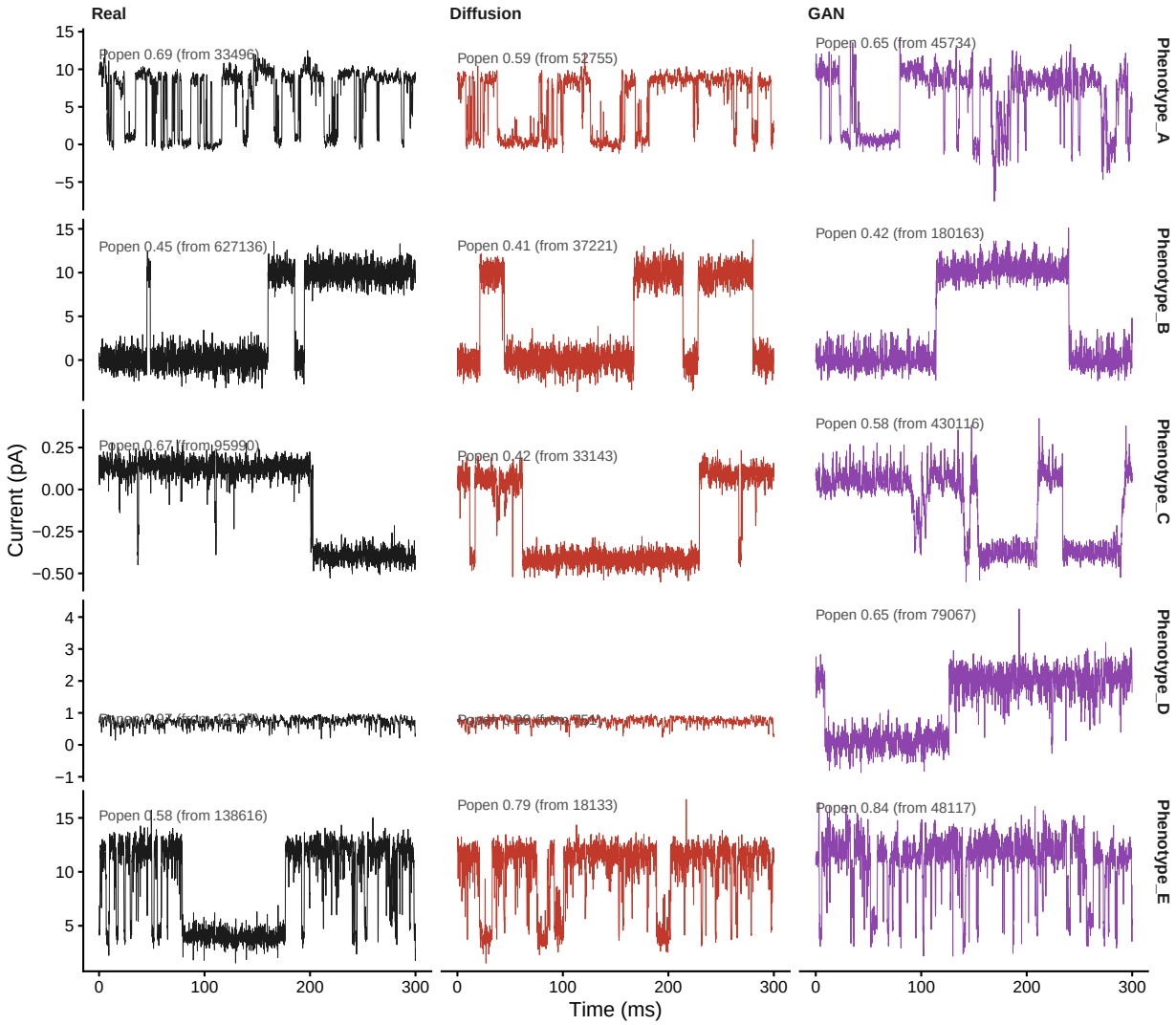
The paired t-test above assumes the five differences are roughly normal, which five points cannot demonstrate. It is quoted as an effect estimate with an interval rather than as a hypothesis test, and the interval is wide by construction. A metric that comes out inconsistent is a finding in itself: it means the two methods are close enough that recording-specific detail decides the ranking.

6. Publication figures

Figure: example traces

```
traces <- pull_all("traces") |>
  dplyr::mutate(source = factor(source, levels = c("Real", "Diffusion", "GAN")))

# Popen of the window actually shown, so a panel cannot quietly misrepresent the
```

```
# record it came from.
win_lab <- traces |>
  dplyr::group_by(phenotype, source) |>
  dplyr::summarise(win_popen = dplyr::first(win_popen),
                   win_start = dplyr::first(win_start),
                   x = min(time_ms), y = max(current), .groups = "drop") |>
  dplyr::mutate(label = sprintf("Popen %.2f (from %d)", win_popen, win_start))

p_traces <- ggplot(traces, aes(time_ms, current, colour = source)) +
  geom_line(linewidth = 0.2) +
  geom_text(data = win_lab, aes(x, y, label = label), hjust = 0, vjust = 1,
            size = 2.4, colour = "grey30", inherit.aes = FALSE) +
  facet_grid(phenotype ~ source, scales = "free_y") +
  scale_colour_manual(values = pal, guide = "none") +
  labs(x = "Time (ms)", y = "Current (pA)") +
  theme_channel()
p_traces
```

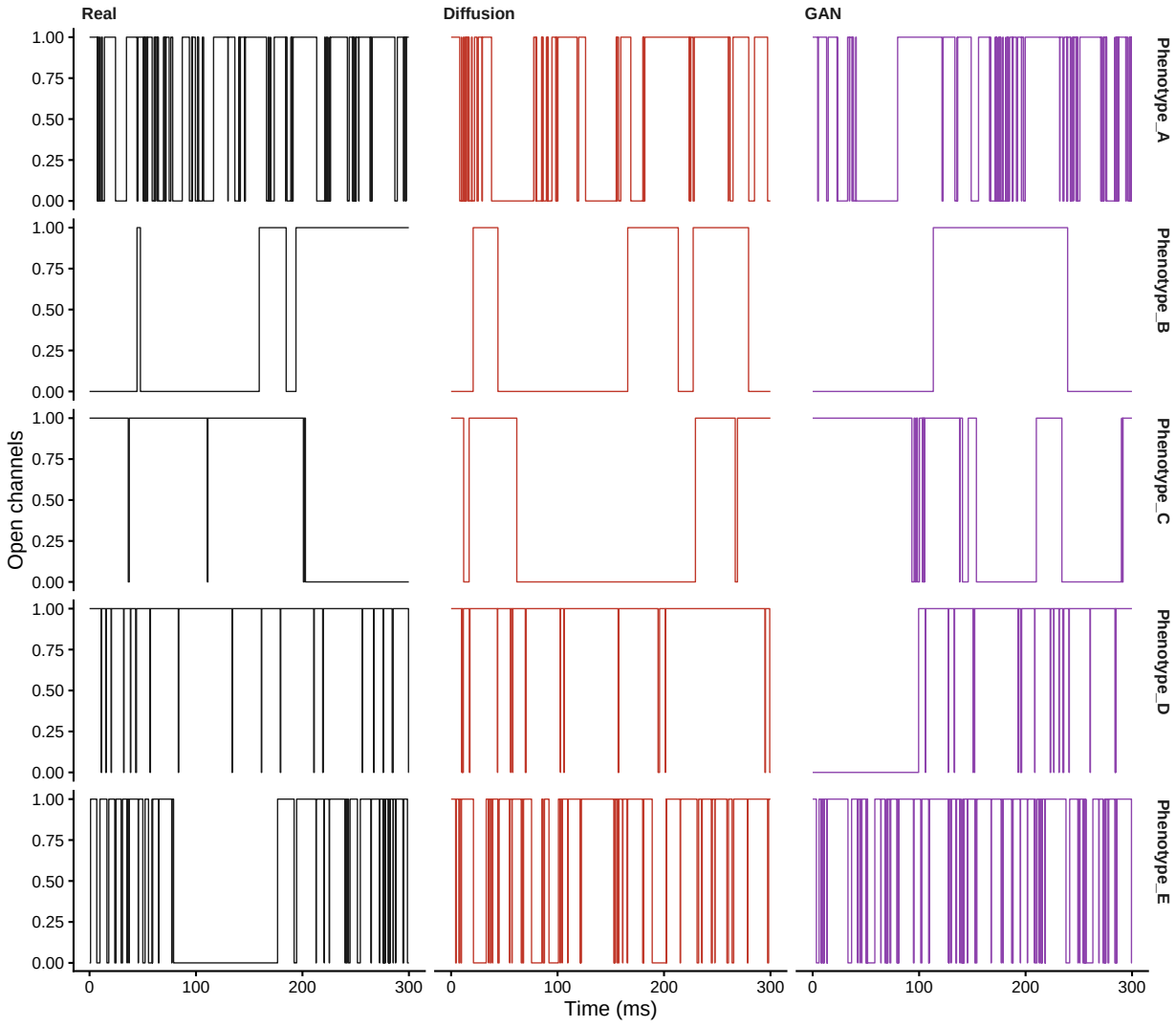


```r
save_fig(p_traces, "fig_traces", width = 8, height = 1.4 * n_ph + 1)
```

The window is chosen by the same rule the Python previews use, so a panel shows gating rather than whichever stretch the file happens to start with. Where no window falls inside the target band — Phenotype_B is too quiet, Phenotype_D too busy — the nearest available is used, and its Popen is printed so the gap is visible rather than implied.

Figure: idealisations

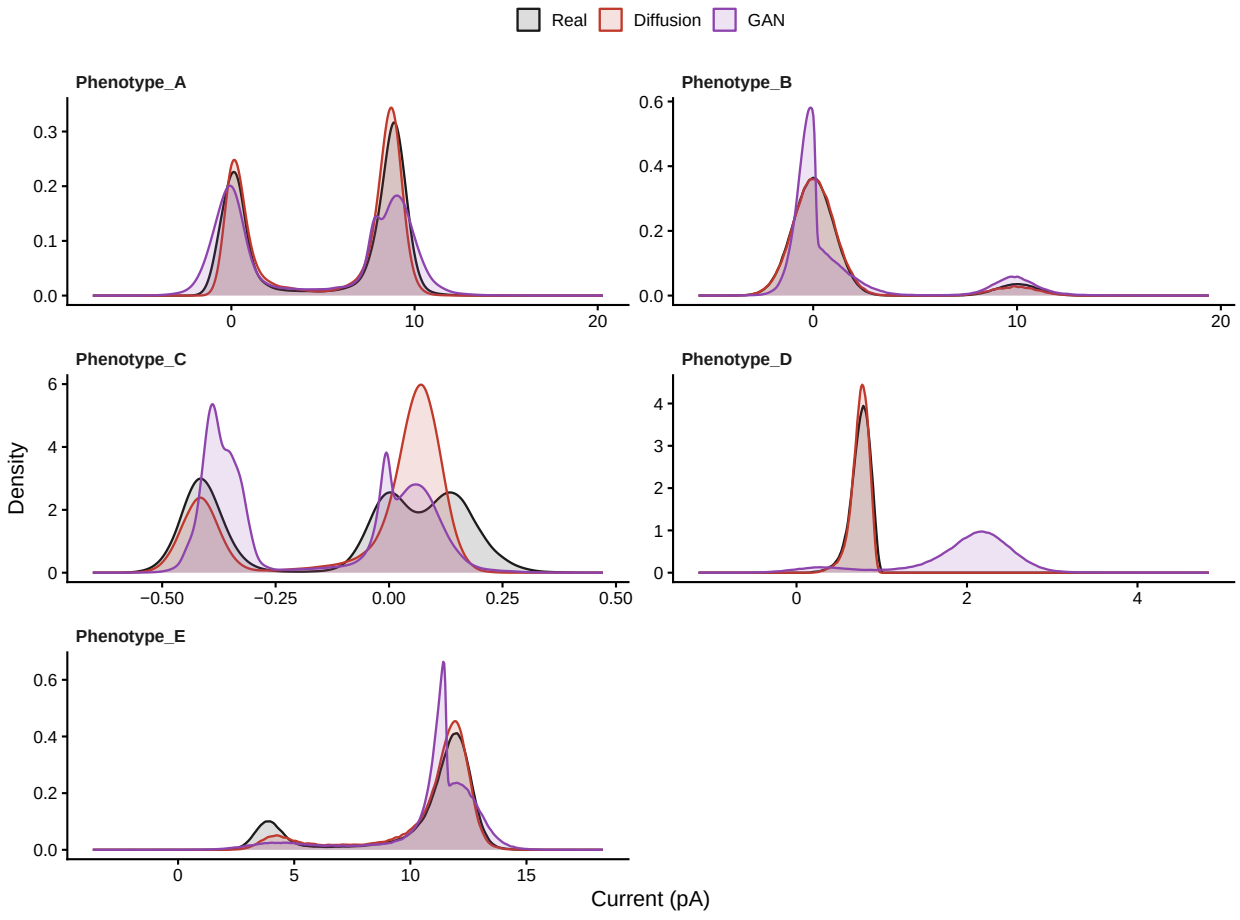
```r
p_ideal <- ggplot(traces, aes(time_ms, channels, colour = source)) +
  geom_step(linewidth = 0.3) +
  facet_grid(phenotype ~ source) +
  scale_colour_manual(values = pal, guide = "none") +
  labs(x = "Time (ms)", y = "Open channels") +
  theme_channel()
p_ideal
```



```
save_fig(p_ideal, "fig_idealisations", width = 8, height = 1.2 * n_ph + 1)
```

Figure: all-points amplitude histograms

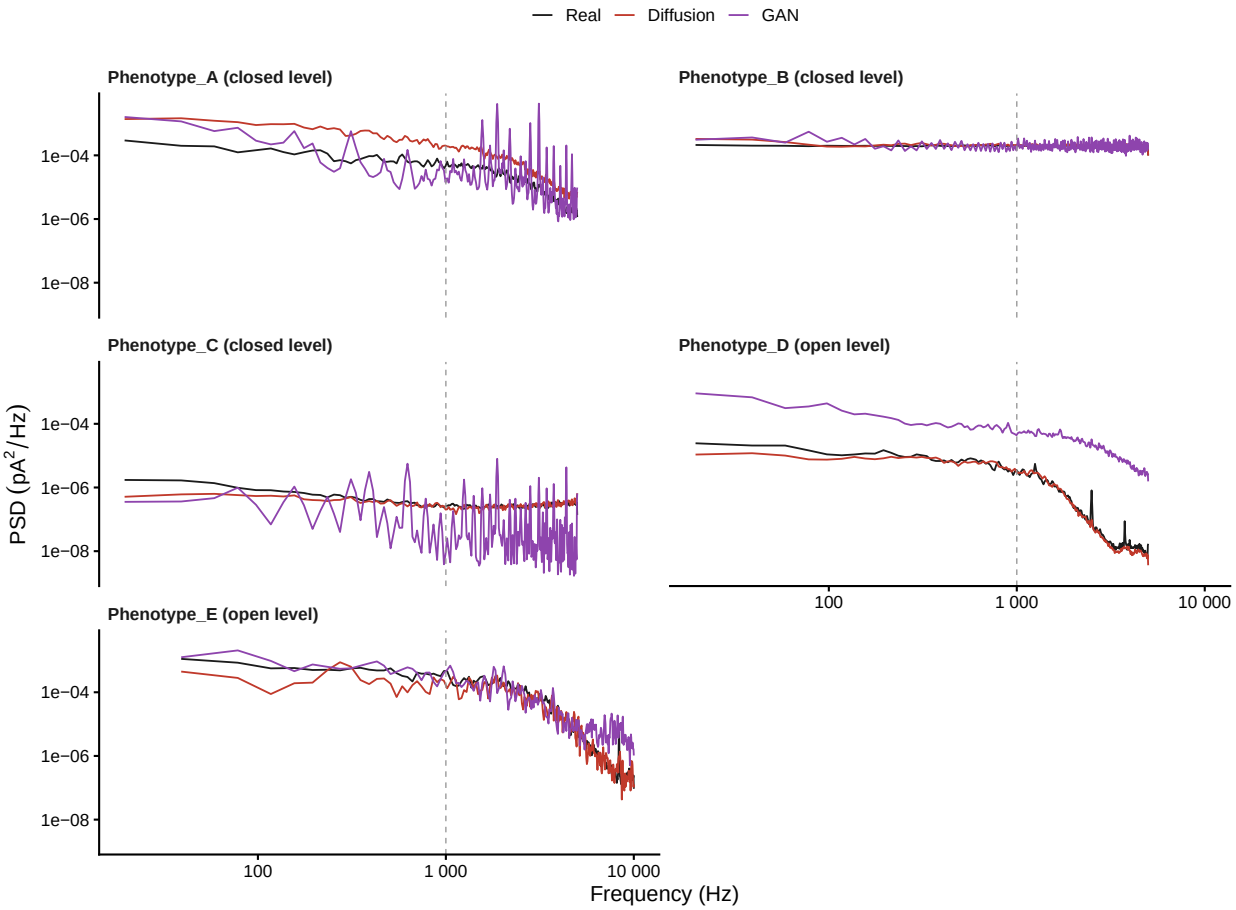
```
p_amp <- pull_all("amp") |>
  dplyr::mutate(source = factor(source, levels = c("Real", "Diffusion", "GAN"))) |>
  ggplot(aes(current, colour = source, fill = source)) +
  geom_density(alpha = 0.15, linewidth = 0.5, adjust = 0.6) +
  facet_wrap(~ phenotype, scales = "free", ncol = 2) +
  scale_colour_manual(values = pal) + scale_fill_manual(values = pal) +
  labs(x = "Current (pA)", y = "Density") +
  theme_channel()
p_amp
```



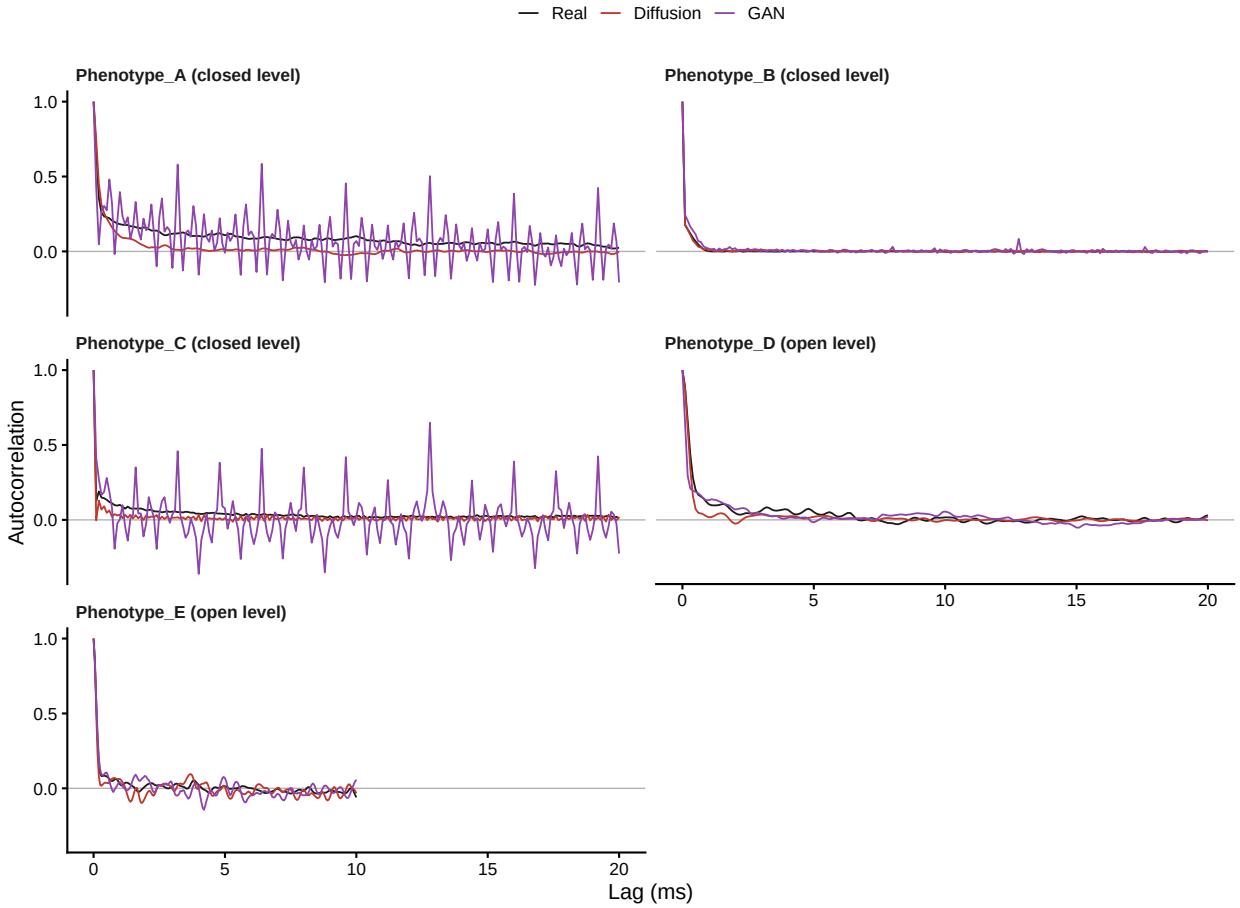

```
save_fig(p_amp, "fig_amplitude", width = 8, height = 2.2 * ceiling(n_ph / 2) + 1)
```

Figure: single-level noise

```r
# Panels are labelled with the level they were computed at, so the legend can
# state it and a phenotype that offers no baseline is not silently different.
panel_label <- setNames(sprintf("%s (%s level)", names(LEVEL_OF), LEVEL_OF),
                        names(LEVEL_OF))

p_psd <- pull_all("psd") |>
  dplyr::mutate(source = factor(source, levels = c("Real", "Diffusion", "GAN")),
                phenotype = panel_label[phenotype]) |>
  ggplot(aes(freq, psd, colour = source)) +
  geom_line(linewidth = 0.4) +
  geom_vline(xintercept = CFG$filter_hz, linetype = "dashed",
             colour = "grey60", linewidth = 0.3) +
  facet_wrap(~ phenotype, ncol = 2) +
  scale_x_log10(labels = scales::label_number()) +
  scale_y_log10(labels = scales::label_scientific()) +
  scale_colour_manual(values = pal) +
  labs(x = "Frequency (Hz)", y = expression(PSD~(pA^2/Hz))) +
  theme_channel()

p_acf <- pull_all("acf") |>
  dplyr::mutate(source = factor(source, levels = c("Real", "Diffusion", "GAN")),
                phenotype = panel_label[phenotype]) |>
  ggplot(aes(lag_ms, acf, colour = source)) +
  geom_hline(yintercept = 0, colour = "grey70", linewidth = 0.3) +
  geom_line(linewidth = 0.4) +
  facet_wrap(~ phenotype, ncol = 2) +
  scale_colour_manual(values = pal) +
  labs(x = "Lag (ms)", y = "Autocorrelation") +
  theme_channel()

p_psd
```


Real
Diffusion
GAN
Phenotype_A (closed level)
Phenotype_B (closed level)
Phenotype_C (closed level)
Phenotype_D (open level)
Phenotype_E (open level)
PSD (pA²/Hz)
1e−04
1e−06
1e−08
100
1 000
10 000
Frequency (Hz)

```
p_acf
```

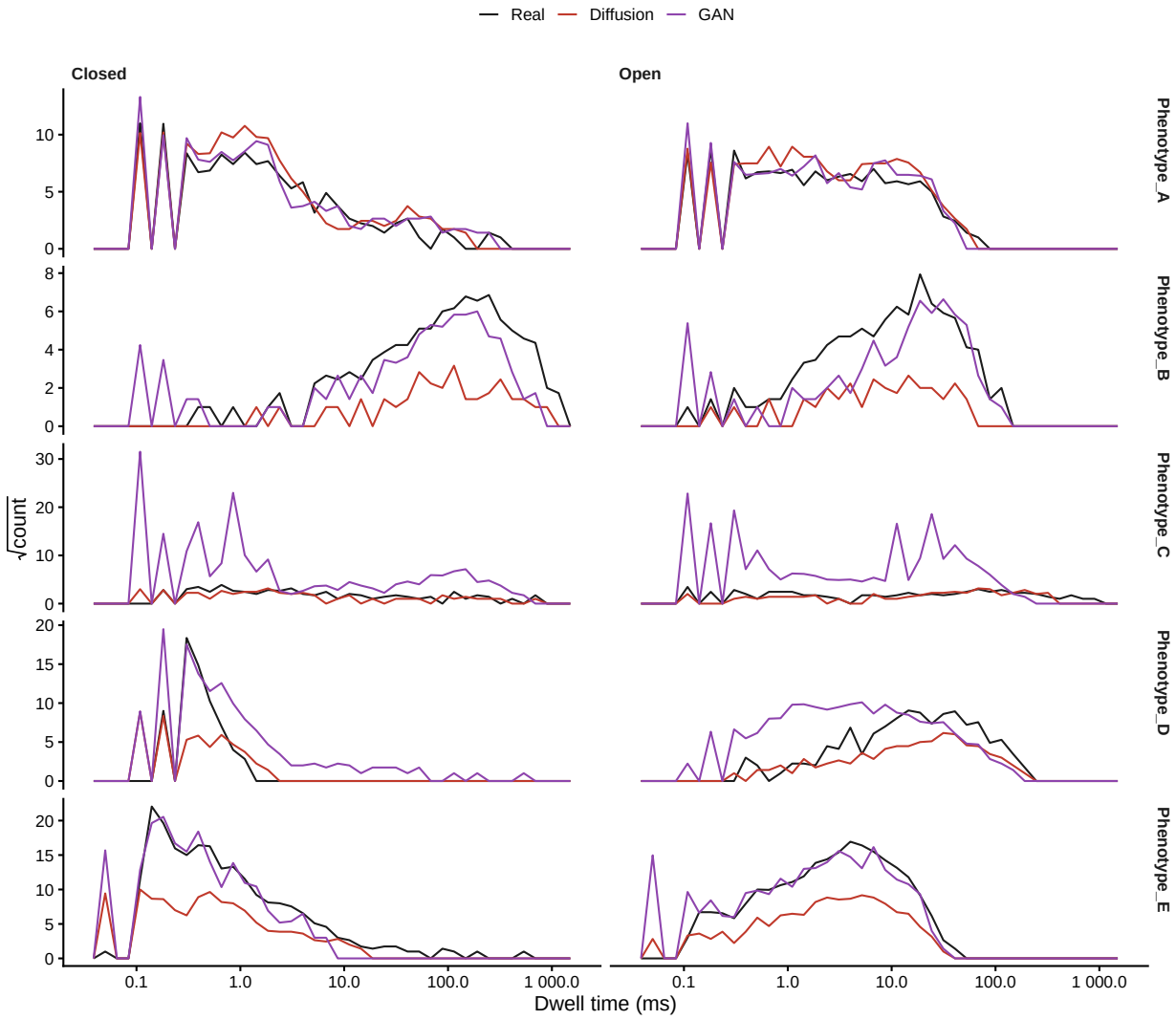



```
save_fig(p_psd, "fig_psd", width = 8, height = 2.2 * ceiling(n_ph / 2) + 1)
save_fig(p_acf, "fig_acf", width = 8, height = 2.2 * ceiling(n_ph / 2) + 1)
```

Figure: dwell times

Sigworth-Sine display: log abscissa, square-root ordinate. The diffusion arm's `Channels` column is the conditioning idealisation it was handed, so it is a known ground truth. The GAN emitted its own labels, so those are a network output. They are plotted together for comparison and must not be described as equivalent in the manuscript.

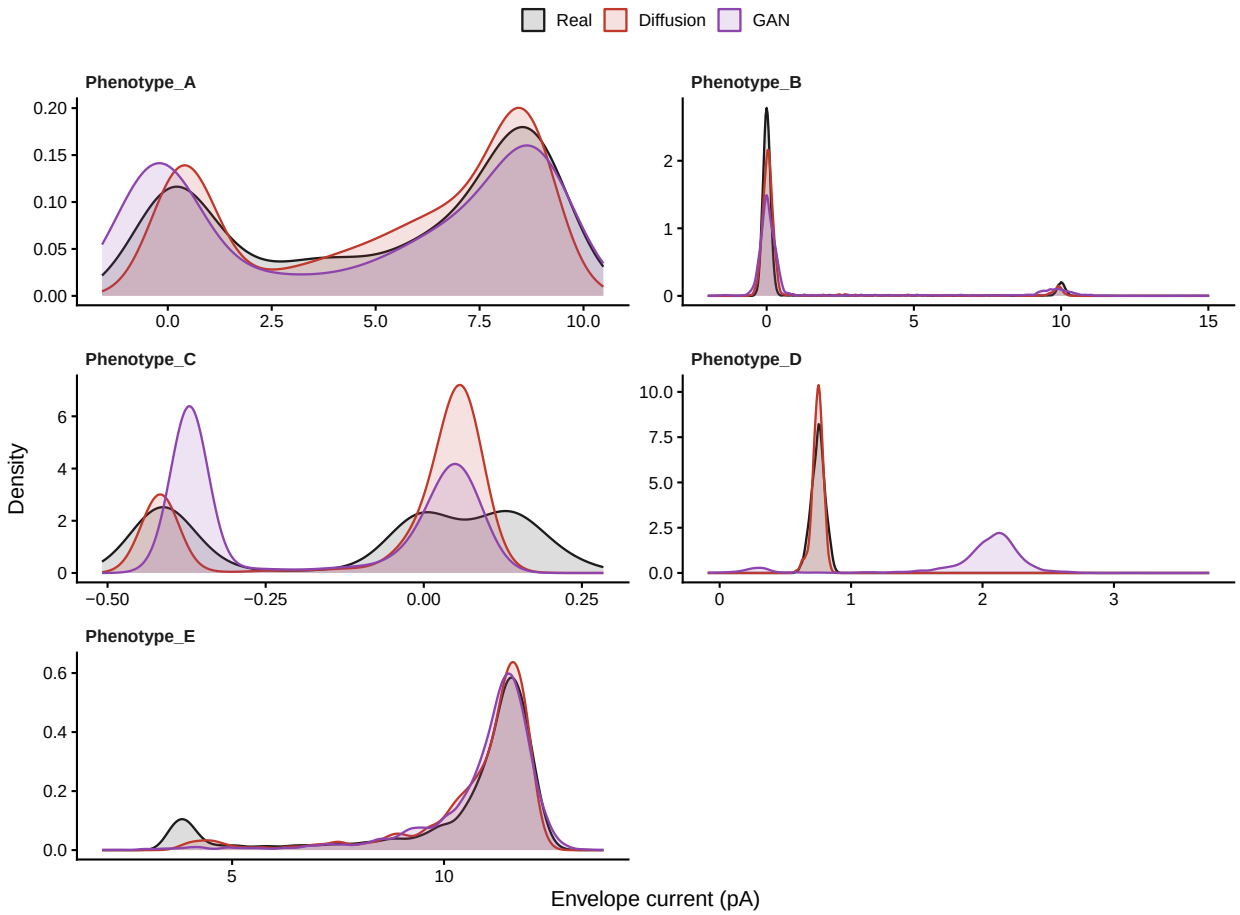
```
p_dwell <- pull_all("dwell") |>
  dplyr::filter(dwell_ms > 0) |>
  dplyr::mutate(source = factor(source, levels = c("Real", "Diffusion", "GAN"))) |>
  ggplot(aes(dwell_ms, colour = source)) +
  geom_freqpoly(aes(y = after_stat(sqrt(count))), bins = 40, linewidth = 0.45) +
  facet_grid(phenotype ~ state, scales = "free_y") +
  scale_x_log10(labels = scales::label_number()) +
  scale_colour_manual(values = pal) +
  labs(x = "Dwell time (ms)", y = expression(sqrt(count))) +
  theme_channel()
p_dwell
```




```
save_fig(p_dwell, "fig_dwell", width = 8, height = 1.5 * n_ph + 1)
```

Figure: slow envelope

```
p_env <- pull_all("env") |>
  dplyr::mutate(source = factor(source, levels = c("Real", "Diffusion", "GAN"))) |>
  ggplot(aes(value, colour = source, fill = source)) +
  geom_density(alpha = 0.15, linewidth = 0.5) +
  facet_wrap(~ phenotype, scales = "free", ncol = 2) +
  scale_colour_manual(values = pal) + scale_fill_manual(values = pal) +
  labs(x = "Envelope current (pA)", y = "Density") +
  theme_channel()
p_env
```




```
save_fig(p_env, "fig_envelope", width = 8, height = 2.2 * ceiling(n_ph / 2) + 1)
```

Figure: manifold projections (optional)

```
if (!all(vapply(c("uwot", "Rtsne"), requireNamespace, logical(1), quietly = TRUE))) {
  stop("Set CFG$do_manifold = FALSE or install uwot and Rtsne.")
}

proj <- purrr::map_dfr(seq_len(nrow(manifest)), function(i) {
  row <- manifest[i, ]
  d <- list(Real = read_record(row$raw), Diffusion = read_record(row$ddpm))
  if (!is.na(row$gan)) d$GAN <- read_record(row$gan)
  Wp <- lapply(d, function(x) make_windows(x$current, CFG$proj_win, CFG$n_windows_proj))
  X <- do.call(rbind, Wp)
  lab <- rep(names(Wp), vapply(Wp, nrow, integer(1)))
  X <- scale(X); X[!is.finite(X)] <- 0
  um <- uwot::umap(X, n_neighbors = 15, min_dist = 0.1, n_components = 2)
  ts <- Rtsne::Rtsne(X, dims = 2, perplexity = 30, check_duplicates = FALSE,
                     pca = TRUE)$Y
  dplyr::bind_rows(
    tibble::tibble(x = um[, 1], y = um[, 2], source = lab, method = "UMAP"),
```

```
    tibble::tibble(x = ts[, 1], y = ts[, 2], source = lab, method = "t-SNE")
  ) |> dplyr::mutate(phenotype = row$phenotype)
})

p_proj <- proj |>
  dplyr::mutate(source = factor(source, levels = c("Real", "Diffusion", "GAN"))) |>
  ggplot(aes(x, y, colour = source)) +
  geom_point(size = 0.5, alpha = 0.5) +
  facet_grid(phenotype ~ method, scales = "free") +
  scale_colour_manual(values = pal) +
  labs(x = NULL, y = NULL) +
  theme_channel() +
  theme(axis.text = element_blank(), axis.ticks = element_blank())
p_proj
save_fig(p_proj, "fig_manifold", width = 7, height = 2 * n_ph + 1)
readr::write_csv(proj, file.path(CFG$out_dir, "manifold_coordinates.csv"))
```

7. Numbers for the manuscript

Rows keyed to `[[placeholder]]` names. Pooled values are medians across phenotypes, with the range, because five records do not support a mean and an interval being read as an estimate.

```
fmt <- function(x, d = 4) formatC(x, digits = d, format = "g")

pooled <- scored |>
  dplyr::group_by(metric, method) |>
  dplyr::summarise(median = median(value), lo = min(value), hi = max(value),
                   .groups = "drop")

placeholders <- dplyr::bind_rows(
  tibble::tibble(placeholder = "n_phenotypes", value = as.character(n_ph),
                 note = "paired seed records"),
  tibble::tibble(placeholder = "n_samples_total",
                 value = format(sum(info$n_real), big.mark = ","),
                 note = "seed samples across all phenotypes"),
  tibble::tibble(placeholder = "sample_khz",
                 value = paste(sort(unique(info$fs_real)) / 1000, collapse = ", "),
                 note = "acquisition rate(s), kHz"),
  pooled |>
    dplyr::transmute(
      placeholder = paste0(gsub("[^A-Za-z0-9]+", "_", tolower(metric)), "_",
                           tolower(method)),
      value = sprintf("%s (%s to %s)", fmt(median), fmt(lo), fmt(hi)),
      note = paste("median across phenotypes, range in brackets")),
  paired_stats |>
    dplyr::transmute(
      placeholder = paste0("unanimity_", gsub("[^A-Za-z0-9]+", "_", tolower(metric))),
      value = sprintf("%d/%d favour Diffusion", n_favour_diff, n),
      note = verdict)
)

knitr::kable(placeholders, caption = "Values keyed to the manuscript placeholders.")
```

Table 13: Values keyed to the manuscript placeholders.

placeholder	value	note
n_phenotypes	5	paired seed records
n_samples_total	1,846,941	seed samples across all phenotypes
sample_khz	10, 10.0000000000233, 20.0000000000466	acquisition rate(s), kHz
acf_rms_diffusion	0.03392 (0.004433 to 0.07455)	median across phenotypes, range in brackets
acf_rms_gan	0.04745 (0.01416 to 0.1447)	median across phenotypes, range in brackets
envelope_sd_pa___diffusion	1.728 (0.04299 to 3.374)	median across phenotypes, range in brackets
envelope_sd_pa___gan	1.465 (0.2049 to 3.998)	median across phenotypes, range in brackets
ks_amplitude_diffusion	0.065 (0.01639 to 0.16)	median across phenotypes, range in brackets
ks_amplitude_gan	0.1178 (0.09704 to 0.8886)	median across phenotypes, range in brackets
mmd_envelope_diffusion	0.009919 (0.001381 to 0.03522)	median across phenotypes, range in brackets
mmd_envelope_gan	0.03671 (0.02497 to 0.9778)	median across phenotypes, range in brackets
mmd_permutation_p_diffusion	0.004975 (0.004975 to 0.3234)	median across phenotypes, range in brackets
mmd_permutation_p_gan	0.004975 (0.004975 to 0.00995)	median across phenotypes, range in brackets
mmd_vs_real_diffusion	0.006285 (0.0001772 to 0.03907)	median across phenotypes, range in brackets
mmd_vs_real_gan	0.01028 (0.002696 to 0.3384)	median across phenotypes, range in brackets
mean_closed_dwell_ms___diffusion	3.601 (0.3932 to 161.6)	median across phenotypes, range in brackets
mean_closed_dwell_ms___gan	4.485 (0.5338 to 124)	median across phenotypes, range in brackets
mean_open_dwell_ms___diffusion	14.11 (4.815 to 78.63)	median across phenotypes, range in brackets
mean_open_dwell_ms___gan	9.924 (4.16 to 22.57)	median across phenotypes, range in brackets
noise_sd_pa___diffusion	0.999 (0.04066 to 1.436)	median across phenotypes, range in brackets
noise_sd_pa___gan	1.12 (0.05601 to 1.551)	median across phenotypes, range in brackets
open_probability_diffusion	0.7548 (0.079 to 0.9879)	median across phenotypes, range in brackets
open_probability_gan	0.5262 (0.154 to 0.8863)	median across phenotypes, range in brackets
log10_psd_rms_diffusion	0.2734 (0.1018 to 0.7906)	median across phenotypes, range in brackets
log10_psd_rms_gan	0.5511 (0.1717 to 1.295)	median across phenotypes, range in brackets
unanimity_acf_rms	5/5 favour Diffusion	consistent: Diffusion
unanimity_envelope_sd_pa_	4/5 favour Diffusion	inconsistent across phenotypes

placeholder	value	note
unanimity_ks_amplitude	5/5 favour Diffusion	consistent: Diffusion
unanimity_mmd_envelope	5/5 favour Diffusion	consistent: Diffusion
unanimity_mmd_permutation_p	2/5 favour Diffusion	inconsistent across phenotypes
unanimity_mmd_vs_real	5/5 favour Diffusion	consistent: Diffusion
unanimity_mean_closed_dwell_ms	5/5 favour Diffusion	consistent: Diffusion
unanimity_mean_open_dwell_ms	3/5 favour Diffusion	inconsistent across phenotypes
unanimity_noise_sd_pa	5/5 favour Diffusion	consistent: Diffusion
unanimity_open_probability	5/5 favour Diffusion	consistent: Diffusion
unanimity_log10_psd_rms	3/5 favour Diffusion	inconsistent across phenotypes

```
readr::write_csv(placeholders, file.path(CFG$out_dir, "results_values.csv"))
cat("\nWritten to", normalizePath(CFG$out_dir), "\n")
```

```
##
## Written to \\wsl.localhost\Ubuntu\home\rbj\LGitHub\DeepGANnel\benchmark_out
```

What this comparison does and does not settle

Both methods are scored identically above, so the numbers are comparable as measures of how much the synthetic data resemble the real record. That is the only question they answer.

They do not settle the question the paper is about. The GAN emits its raw trace and its `Channels` column together, so its labels are a network output; the diffusion pipeline is handed its labels before it starts and never touches them. A GAN record could match the real data better on every metric here and still be the less useful training set, because a downstream idealiser trained against it is being scored on another network's opinion rather than on truth. If the GAN wins on similarity, say so plainly and make the ground-truth argument on its own terms.

Session information

```
sessionInfo()
```

```
## R version 4.5.2 (2025-10-31 ucrt)
## Platform: x86_64-w64-mingw32/x64
## Running under: Windows 11 x64 (build 26200)
##
## Matrix products: default
##   LAPACK version 3.12.1
##
## locale:
## [1] LC_COLLATE=English_United Kingdom.utf8
## [2] LC_CTYPE=English_United Kingdom.utf8
## [3] LC_MONETARY=English_United Kingdom.utf8
## [4] LC_NUMERIC=C
## [5] LC_TIME=English_United Kingdom.utf8
##
## time zone: Europe/London
## tzcode source: internal
```

```
##
## attached base packages:
## [1] stats     graphics  grDevices utils     datasets  methods   base
##
## other attached packages:
## [1] patchwork_1.3.2 arrow_25.0.0    purrr_1.2.2     readr_2.2.0
## [5] ggplot2_4.0.3   tidyr_1.3.2     dplyr_1.2.1
##
## loaded via a namespace (and not attached):
##  [1] bit_4.6.0          gtable_0.3.6       crayon_1.5.3       compiler_4.5.2
##  [5] tidyselect_1.2.1   parallel_4.5.2     assertthat_0.2.1   scales_1.4.0
##  [9] yaml_2.3.12        fastmap_1.2.0      R6_2.6.1           labeling_0.4.3
## [13] generics_0.1.4     knitr_1.51         tibble_3.3.1       pillar_1.11.1
## [17] RColorBrewer_1.1-3 tzdb_0.5.0         rlang_1.3.0        xfun_0.60
## [21] S7_0.2.2           bit64_4.8.2        cli_3.6.6          withr_3.0.3
## [25] magrittr_2.0.5     digest_0.6.39      grid_4.5.2         vroom_1.7.1
## [29] hms_1.1.4          lifecycle_1.0.5    vctrs_0.7.3        evaluate_1.0.5
## [33] glue_1.8.1         farver_2.1.2       rmarkdown_2.31     tools_4.5.2
## [37] pkgconfig_2.0.3    htmltools_0.5.9
```